BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC [TÊN TRƯỜNG]
KHOA CÔNG NGHỆ THÔNG TIN

———————— ✦ ————————

BÁO CÁO ĐỒ ÁN MÔN HỌC

ĐIỆN TOÁN ĐÁM MÂY

ĐỀ TÀI

DESIGN AND DEPLOYMENT OF A SPRING BOOT CONTAINERIZED CLOUD FILE STORAGE PLATFORM BASED ON AWS S3-COMPATIBLE ARCHITECTURE WITH AN AUTOMATED CI/CD PIPELINE

(Thiết kế và triển khai nền tảng lưu trữ file đám mây dạng container bằng Spring Boot theo kiến trúc tương thích AWS S3, tích hợp quy trình CI/CD tự động)

Sản phẩm: NIMBUS DRIVE

Giảng viên hướng dẫn:	[Họ tên giảng viên]
Sinh viên thực hiện:	[Họ tên sinh viên]
Mã số sinh viên:	[MSSV]
Lớp:	[Lớp học phần]

[Thành phố], tháng 11 năm 2026

LỜI CẢM ƠN

Em xin chân thành cảm ơn [giảng viên hướng dẫn] đã tận tình giảng dạy học phần Điện toán đám mây và định hướng cho em trong suốt quá trình thực hiện đồ án. Những kiến thức về mô hình dịch vụ đám mây, lưu trữ đối tượng, container hóa và tự động hóa triển khai là nền tảng để em xây dựng sản phẩm Nimbus Drive.

Do thời gian và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi thiếu sót. Em rất mong nhận được góp ý của thầy/cô để hoàn thiện sản phẩm hơn.

Sinh viên thực hiện

[Họ tên sinh viên]

MỤC LỤC

LỜI CẢM ƠN 2
MỤC LỤC 3
DANH MỤC HÌNH ẢNH 6
DANH MỤC TỪ VIẾT TẮT 8
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 9
1.1. Lý do chọn đề tài 9
1.2. Mục tiêu 9
1.3. Phạm vi và đối tượng 9
1.4. Phương pháp thực hiện 10
1.5. Khảo sát các giải pháp hiện có 10
1.6. Bố cục báo cáo 11
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 12
2.1. Điện toán đám mây 12
2.2. Lưu trữ đối tượng và giao thức Amazon S3 12
2.3. Container hóa với Docker 13
2.4. Tích hợp và triển khai liên tục (CI/CD) 14
2.5. Hạ tầng dưới dạng mã (Infrastructure as Code) 14
2.6. Các công nghệ phía ứng dụng 14
2.7. Xác thực bằng JWT và băm mật khẩu BCrypt 15
2.8. Quản lý schema, kiểm thử và giám sát 15
2.9. Presigned URL so với tải qua máy chủ ứng dụng 15
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 17
3.1. Yêu cầu chức năng 17
3.2. Yêu cầu phi chức năng 18
3.3. Kiến trúc tổng thể 18
3.4. Kiến trúc phần mềm 19
3.5. Thiết kế cơ sở dữ liệu 20
3.6. Thiết kế API 21

3.7. Luồng xử lý upload và tải qua presigned URL 23
3.8. Thiết kế bảo mật 23
3.8.1. Luồng xác thực JWT 24
3.8.2. Vòng đời link chia sẻ 25
3.9. Các quyết định thiết kế quan trọng 25
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG 27
4.1. Cấu trúc mã nguồn 27
4.2. Tầng lưu trữ tương thích S3 27
4.2.1. Thứ tự ghi dữ liệu khi upload và xóa 27
4.2.2. Kiểm soát quyền sở hữu, quota và tên file 28
4.3. Giao diện người dùng 28
4.4. Container hóa 29
4.4.1. Dockerfile nhiều giai đoạn 29
4.4.2. Docker Compose 30
4.5. Pipeline CI/CD với GitHub Actions 31
4.6. Hạ tầng AWS bằng Terraform 32
4.7. Giám sát 33
4.8. Quy trình triển khai, nâng cấp và rollback 34
CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ 35
5.1. Giao diện ứng dụng 35
5.2. Kiểm thử 41
5.3. Triển khai thực tế trên AWS 42
5.4. Đánh giá 43
Ưu điểm 43
Hạn chế 43
5.5. Các sự cố gặp phải và cách xử lý 44
5.6. Chi phí vận hành 45
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 46
6.1. Kết luận 46
6.2. Hướng phát triển 46

6.3. Bài học kinh nghiệm.....46
TÀI LIỆU THAM KHẢO.....48
PHỤ LỤC: HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY 49

DANH MỤC HÌNH ẢNH

Bảng 0.1. Các từ viết tắt 8
Bảng 1.1. So sánh Nimbus Drive với các giải pháp khảo sát 10
Bảng 2.1. Các mô hình dịch vụ và vị trí của Nimbus Drive 12
Bảng 2.2. So sánh ba loại lưu trữ 12
Bảng 2.3. Khái niệm cốt lõi của Amazon S3 và cách đề tài sử dụng 13
Bảng 2.4. Công nghệ sử dụng 14
Bảng 2.5. Hai cách phục vụ nội dung file 15
Hình 3.1. Sơ đồ use case tổng quát 17
Bảng 3.1. Đặc tả một số use case chính 17
Bảng 3.2. Yêu cầu chức năng 17
Hình 3.2. Kiến trúc tổng thể hệ thống Nimbus Drive 18
Hình 3.3. Kiến trúc phân lớp và mẫu Ports & Adapters cho tầng lưu trữ 19
Hình 3.4. Lược đồ quan hệ thực thể (ERD) 20
Bảng 3.3. Mô tả các bảng 20
Bảng 3.4. Từ điển dữ liệu – bảng users và folders 20
Bảng 3.5. Từ điển dữ liệu – bảng files và share_links 21
Bảng 3.6. Các endpoint REST chính 21
Bảng 3.7. Bảng mã trạng thái HTTP của API 22
Hình 3.5. Sơ đồ tuần tự upload file và tải trực tiếp bằng presigned URL 23
Bảng 3.8. Các biện pháp bảo mật 23
Hình 3.6. Mô hình phòng thủ nhiều lớp (defense in depth) 24
Hình 3.7. Luồng đăng nhập và gọi API bằng Bearer token 24
Hình 3.8. Các trạng thái của link chia sẻ công khai 25
Bảng 3.9. Bảng quyết định kiến trúc (ADR rút gọn) 25
Bảng 4.1. Các tệp giao diện và trách nhiệm 29
Bảng 4.2. Các service trong docker-compose.yml 30
Bảng 4.3. Các biến môi trường cấu hình (nguyên tắc 12-factor) 30
Hình 4.1. Pipeline CI/CD tự động 31

Bảng 4.4. Các job trong pipeline 31
Bảng 4.5. Cấu hình GitHub cần thiết cho bước deploy 31
Hình 4.2. Kiến trúc triển khai trên AWS 32
Bảng 4.6. Tài nguyên Terraform tạo ra 32
Bảng 4.7. Biến Terraform chính 32
Bảng 4.8. Các biểu đồ trên dashboard Grafana 33
Bảng 4.9. Quy trình triển khai từ đầu tới cuối 34
Hình 5.1. Trang đăng nhập / đăng ký 35
Hình 5.2. Trang quản lý file – chế độ lưới 35
Hình 5.3. Thư mục ảnh với ảnh thu nhỏ 36
Hình 5.4. Xem trước ảnh trực tiếp 36
Hình 5.5. Chế độ danh sách và breadcrumb thư mục lồng nhau 37
Hình 5.6. Menu thao tác trên file 37
Hình 5.7. Hộp thoại tạo link chia sẻ có thời hạn 38
Hình 5.8. Quản lý các link đã chia sẻ 38
Hình 5.9. Giao diện tối (dark mode) 39
Hình 5.10. Trang tải file công khai cho khách 39
Hình 5.11. Tài liệu API Swagger UI 40
Hình 5.12. Giao diện trên điện thoại 40
Bảng 5.1. Kết quả kiểm thử tự động (mvn verify) 41
Bảng 5.2. Danh sách 22 ca kiểm thử 41
Bảng 5.3. Kết quả kiểm thử trên môi trường AWS 42
Bảng 5.4. Đối chiếu mục tiêu 43
Bảng 5.5. Nhật ký sự cố và bài học 44
Bảng 5.6. Các thành phần chi phí khi chạy trên AWS 45
Bảng 6.1. Đề xuất phát triển tiếp 46
Bảng 6.2. Bảng xử lý sự cố 49
Bảng 6.3. Cổng dịch vụ 50

DANH MỤC TỪ VIẾT TẮT

Bảng 0.1. Các từ viết tắt

Viết tắt	Ý nghĩa
API	Application Programming Interface – giao diện lập trình ứng dụng
AWS	Amazon Web Services
S3	Amazon Simple Storage Service – dịch vụ lưu trữ đối tượng
CI/CD	Continuous Integration / Continuous Delivery (Deployment)
IaaS / PaaS / SaaS	Infrastructure / Platform / Software as a Service
IaC	Infrastructure as Code – hạ tầng dưới dạng mã
JWT	JSON Web Token
JPA	Jakarta Persistence API
IAM	Identity and Access Management
EC2	Amazon Elastic Compute Cloud
GHCR	GitHub Container Registry
SigV4	AWS Signature Version 4 – cơ chế ký request của S3
CSP	Content Security Policy
XSS	Cross-Site Scripting

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Nhu cầu lưu trữ và chia sẻ dữ liệu trực tuyến tăng mạnh cùng với sự phổ biến của làm việc từ xa và học tập trực tuyến. Các dịch vụ như Google Drive, Dropbox hay OneDrive đều được xây dựng trên nền tảng **lưu trữ đối tượng (object storage)** – loại lưu trữ có khả năng mở rộng gần như vô hạn, độ bền dữ liệu cao và chi phí thấp. Trong đó, giao thức **Amazon S3** đã trở thành chuẩn de-facto: hầu hết nhà cung cấp (Google Cloud Storage, Cloudflare R2, DigitalOcean Spaces, Ceph, SeaweedFS, MinIO…) đều cung cấp API tương thích S3.

Bên cạnh đó, phương thức phát triển phần mềm hiện đại đòi hỏi ứng dụng được **đóng gói thành container** và **triển khai tự động** qua pipeline CI/CD để giảm lỗi thủ công, rút ngắn thời gian đưa tính năng tới người dùng. Đề tài kết hợp cả ba trụ cột của điện toán đám mây: dịch vụ lưu trữ đám mây, container hóa và tự động hóa vận hành (DevOps).

1.2. Mục tiêu

- Thiết kế và xây dựng ứng dụng web lưu trữ file **Nimbus Drive** bằng Spring Boot 3, cho phép người dùng đăng ký, tải lên, tổ chức thư mục, xem trước, tìm kiếm và chia sẻ file.
- Tách biệt **metadata** (PostgreSQL) và **nội dung file** (object storage) theo kiến trúc tương thích S3; ứng dụng chạy được trên Amazon S3 thật hoặc storage tự host mà không đổi mã nguồn.
- Đóng gói toàn bộ hệ thống bằng **Docker / Docker Compose**; mô tả hạ tầng AWS bằng **Terraform**.
- Xây dựng pipeline **GitHub Actions** tự động: build → kiểm thử → build image → quét lỗ hổng → đẩy registry → triển khai → smoke test.
- Đảm bảo các yêu cầu phi chức năng: bảo mật, khả năng giám sát (observability), dễ mở rộng.

1.3. Phạm vi và đối tượng

Đối tượng sử dụng là cá nhân hoặc nhóm nhỏ cần kho lưu trữ riêng (self-hosted) hoặc triển khai trên AWS. Phạm vi đồ án gồm: backend REST API, giao diện web SPA, cơ sở dữ liệu, lớp lưu trữ đối tượng, đóng gói container, hạ tầng dưới dạng mã và pipeline CI/CD. Không bao gồm ứng dụng di động native, đồng bộ hai chiều kiểu desktop client và thanh toán.

1.4. Phương pháp thực hiện

- Nghiên cứu tài liệu chính thức: AWS S3 Developer Guide, Spring Boot Reference, Docker, GitHub Actions, Terraform AWS Provider.
- Phân tích yêu cầu → thiết kế kiến trúc phân lớp kết hợp mẫu **Ports & Adapters** cho tầng lưu trữ.
- Phát triển lặp, kiểm thử tự động (unit test, API test với MockMvc, integration test với Testcontainers).
- Tự động hóa build/deploy, đo đạc và giám sát bằng Spring Boot Actuator, Prometheus và Grafana.

1.5. Khảo sát các giải pháp hiện có

Trước khi thiết kế, em khảo sát bốn nhóm giải pháp lưu trữ file phổ biến để xác định khoảng trống mà đề tài có thể lấp đầy. Các dịch vụ thương mại như Google Drive hay Dropbox rất tiện dụng nhưng dữ liệu nằm hoàn toàn trong hệ sinh thái của nhà cung cấp, người dùng không kiểm soát được vị trí lưu trữ và không thể tự mở rộng tính năng. Nextcloud cho phép tự host nhưng là một khối ứng dụng PHP lớn, khó dùng để học cách thiết kế kiến trúc cloud-native. Các công cụ dòng lệnh/console của S3 (AWS Console, MinIO Console) lại chỉ là giao diện quản trị bucket, không có khái niệm người dùng, thư mục, quota hay chia sẻ.

Bảng 1.1. So sánh Nimbus Drive với các giải pháp khảo sát

Tiêu chí	**Google Drive / Dropbox**	**Nextcloud**	**S3 Console**	**Nimbus Drive**
Mô hình	SaaS đóng	Tự host (PHP)	Công cụ quản trị	Tự host hoặc AWS, mã nguồn mở
Kiểm soát dữ liệu	Thấp	Cao	Cao	Cao (chọn S3/SeaweedFS)
Quản lý user, quota	Có	Có	Không	Có (JWT, quota 1 GB mặc định)
Thư mục, chia sẻ link	Có	Có	Không	Có (thư mục ảo, link có hạn)
Tải trực tiếp từ object storage	Không công khai	Tùy cấu hình	Có	Có (presigned URL)
Đóng gói container + IaC	Không áp dụng	Có (cộng đồng)	Không áp dụng	Có (Compose + Terraform)
CI/CD mẫu sẵn	Không	Không	Không	Có (GitHub Actions)
Độ phức tạp để học	Không phù hợp	Cao	Thấp	Vừa phải, mã ngắn gọn

Từ khảo sát, đề tài định vị Nimbus Drive là một **hệ thống tham chiếu gọn nhẹ**: đủ tính năng để dùng thật, đủ nhỏ để đọc hiểu toàn bộ mã nguồn (khoảng 2.000 dòng

Java và 1.400 dòng giao diện), và đặt trọng tâm vào các kỹ thuật điện toán đám mây thay vì tính năng cộng tác phức tạp.

1.6. Bố cục báo cáo

- **Chương 1** giới thiệu bài toán, mục tiêu, phạm vi và khảo sát giải pháp hiện có.
- **Chương 2** trình bày cơ sở lý thuyết: điện toán đám mây, lưu trữ đối tượng S3, container, CI/CD, IaC và các công nghệ sử dụng.
- **Chương 3** phân tích yêu cầu và thiết kế kiến trúc, cơ sở dữ liệu, API, luồng xử lý và bảo mật.
- **Chương 4** mô tả chi tiết quá trình triển khai: mã nguồn, container hóa, pipeline CI/CD, hạ tầng AWS và giám sát.
- **Chương 5** trình bày kết quả, kiểm thử, triển khai thực tế, các sự cố gặp phải và đánh giá.
- **Chương 6** kết luận và đề xuất hướng phát triển; phần phụ lục hướng dẫn cài đặt, cấu hình và ví dụ gọi API.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Điện toán đám mây

Theo định nghĩa của NIST (SP 800-145), điện toán đám mây là mô hình cho phép truy cập qua mạng, theo yêu cầu, tới một tập tài nguyên tính toán dùng chung có thể cấu hình (mạng, máy chủ, lưu trữ, ứng dụng, dịch vụ), được cấp phát và giải phóng nhanh chóng với nỗ lực quản lý tối thiểu. Năm đặc trưng cốt lõi: tự phục vụ theo nhu cầu, truy cập mạng rộng rãi, gộp tài nguyên, co giãn nhanh và dịch vụ đo lường được.

Bảng 2.1. Các mô hình dịch vụ và vị trí của Nimbus Drive

Mô hình	**Nhà cung cấp quản lý**	**Áp dụng trong đồ án**
IaaS	Phần cứng, ảo hóa, mạng	Amazon EC2, EBS, VPC, Elastic IP
PaaS / dịch vụ quản lý	Hệ điều hành, runtime, dịch vụ	Amazon S3 (object storage), GHCR, GitHub Actions
SaaS	Toàn bộ ứng dụng	Nimbus Drive cung cấp cho người dùng cuối dưới dạng SaaS

2.2. Lưu trữ đối tượng và giao thức Amazon S3

Lưu trữ đối tượng tổ chức dữ liệu thành các **object** nằm phẳng trong **bucket**, mỗi object gồm dữ liệu, metadata và một **key** duy nhất. Khác với file system phân cấp hay block storage, object storage truy cập qua HTTP API, mở rộng ngang dễ dàng và có độ bền rất cao (Amazon S3 thiết kế cho độ bền 99,999999999%). Các thao tác chính: PutObject, GetObject, DeleteObject, HeadBucket, CreateBucket, ListObjectsV2.

Mọi request S3 được ký bằng **AWS Signature Version 4 (SigV4)** dựa trên access key/secret key. **Presigned URL** là URL đã được ký sẵn kèm thời hạn, cho phép bên thứ ba (ví dụ trình duyệt) tải trực tiếp object mà không cần credential, giúp giảm tải cho máy chủ ứng dụng. Vì S3 là chuẩn mở trên thực tế, ứng dụng viết theo S3 API có thể chuyển giữa AWS và các hệ thống tương thích như SeaweedFS, Ceph RGW, MinIO chỉ bằng cách đổi endpoint.

Trong quá trình thực hiện, nhóm ghi nhận **MinIO bản cộng đồng đã ngừng phát hành bản dựng sẵn** (trang tải về trả mã 410 Gone). Vì vậy đồ án chọn **SeaweedFS** (giấy phép Apache 2.0, vẫn được duy trì tích cực) làm S3 server tự host; nhờ thiết kế trừu tượng hóa, thay đổi này không ảnh hưởng mã nghiệp vụ – một minh chứng cho lợi ích của kiến trúc tương thích S3.

Bảng 2.2. So sánh ba loại lưu trữ

Tiêu chí	**Block storage (EBS)**	**File storage (EFS/NFS)**	**Object storage (S3)**
Đơn vị dữ liệu	Khối (block) gắn như	File trong cây thư mục	Object = dữ liệu +

	ổ đĩa		metadata + key
Truy cập	Qua hệ điều hành của 1 máy	Giao thức NFS/SMB, nhiều máy	HTTP API (REST), mọi nơi
Khả năng mở rộng	Giới hạn theo dung lượng ổ	Lớn nhưng có trần	Gần như vô hạn
Độ trễ	Rất thấp	Thấp	Cao hơn (hàng chục ms)
Chi phí / GB	Cao	Trung bình – cao	Thấp
Phù hợp	CSDL, hệ điều hành	Dùng chung giữa nhiều máy	File người dùng, sao lưu, media, data lake

Vì nhu cầu của đề tài là lưu file do người dùng tải lên (ghi một lần, đọc nhiều lần, kích thước từ vài KB tới hàng trăm MB, số lượng không biết trước), object storage là lựa chọn tối ưu về chi phí và khả năng mở rộng; CSDL quan hệ chỉ giữ phần metadata nhỏ.

Bảng 2.3. Khái niệm cốt lõi của Amazon S3 và cách đề tài sử dụng

Khái niệm	**Ý nghĩa**	**Áp dụng trong Nimbus Drive**
Bucket	Thùng chứa có tên duy nhất toàn cầu, gắn với một vùng (region)	1 bucket cho toàn hệ thống: nimbus-files-<hậu tố ngẫu nhiên>
Object key	Tên duy nhất của object trong bucket (phẳng, "/" chỉ là ký tự)	owner/<uuid> – không chứa tên file gốc để tránh trùng và lộ thông tin
Versioning	Giữ nhiều phiên bản của cùng một key	Bật; lifecycle xóa phiên bản cũ sau 30 ngày
SSE-S3	Mã hóa phía server bằng khóa do S3 quản lý (AES-256)	Bật mặc định bằng Terraform
Block Public Access	Chặn mọi cấu hình làm bucket công khai	Bật cả 4 cờ; truy cập chỉ qua IAM hoặc presigned URL
Presigned URL	URL ký sẵn, tự hết hạn, ủy quyền tạm thời	Tải file 15 phút, kèm Content-Disposition đúng tên file
Lifecycle rule	Tự động chuyển lớp lưu trữ / xóa theo tuổi	Hủy multipart dở sau 7 ngày, xóa phiên bản cũ sau 30 ngày
CORS	Cho phép trình duyệt gọi S3 từ origin khác	Cho phép GET/HEAD để xem trước bằng presigned URL

Về tính nhất quán, từ tháng 12/2020 Amazon S3 đảm bảo **strong read-after-write consistency** cho mọi thao tác PUT/DELETE, nên ứng dụng có thể đọc ngay sau khi ghi mà không cần cơ chế thử lại. Điều này đơn giản hóa luồng upload → hiển thị ảnh thu nhỏ trong giao diện.

2.3. Container hóa với Docker

Container đóng gói ứng dụng cùng toàn bộ thư viện phụ thuộc thành **image** bất biến, chạy cô lập trên nhân hệ điều hành dùng chung nhờ namespaces và cgroups. So với máy ảo, container khởi động nhanh, nhẹ và đảm bảo tính nhất quán "build once,

run anywhere". **Docker Compose** mô tả hệ thống nhiều container (app, database, storage, proxy, giám sát) trong một file YAML, khai báo mạng, volume, healthcheck và thứ tự phụ thuộc.

2.4. Tích hợp và triển khai liên tục (CI/CD)

CI (Continuous Integration) là việc tự động build và kiểm thử mỗi khi có thay đổi mã nguồn để phát hiện lỗi sớm. **CD** (Continuous Delivery/Deployment) tự động đóng gói và đưa phiên bản đã kiểm thử lên môi trường chạy thật. GitHub Actions cung cấp runner được quản lý, định nghĩa pipeline bằng YAML theo sự kiện (push, pull request) và tích hợp sẵn registry GHCR.

2.5. Hạ tầng dưới dạng mã (Infrastructure as Code)

IaC mô tả hạ tầng (máy chủ, mạng, bucket, quyền) bằng mã nguồn có thể review, version và tái tạo. **Terraform** dùng ngôn ngữ khai báo HCL, so sánh trạng thái mong muốn với trạng thái thực tế để lập kế hoạch (plan) và áp dụng (apply) thay đổi.

2.6. Các công nghệ phía ứng dụng

Bảng 2.4. Công nghệ sử dụng

Thành phần	Công nghệ	Vai trò
Ngôn ngữ / framework	Java 21, Spring Boot 3.5	REST API, DI, cấu hình theo môi trường
Bảo mật	Spring Security, JJWT 0.12, BCrypt	Xác thực stateless bằng JWT, băm mật khẩu
Dữ liệu	Spring Data JPA, Hibernate 6, PostgreSQL 16, Flyway	Lưu metadata, quản lý phiên bản schema
Lưu trữ	AWS SDK for Java v2 (S3Client, S3Presigner)	Giao tiếp S3 API, ký presigned URL
Tài liệu API	springdoc-openapi 2.10 (Swagger UI)	Tài liệu và thử API trực tuyến
Giám sát	Actuator, Micrometer, Prometheus, Grafana	Health check, metrics, dashboard
Kiểm thử	JUnit 5, MockMvc, Testcontainers, JaCoCo	Unit/API/integration test, đo độ phủ
Frontend	HTML5, CSS3, JavaScript thuần (ES2020)	SPA không cần bước build, tuân thủ CSP
Hạ tầng	Docker, Compose, Caddy, Terraform, GitHub Actions	Đóng gói, HTTPS, IaC, CI/CD

2.7. Xác thực bằng JWT và băm mật khẩu BCrypt

JSON Web Token (RFC 7519) gồm ba phần mã hóa Base64URL ngăn cách bởi dấu chấm: header (thuật toán, ví dụ HS256), payload (các claim như sub – tên người dùng, iat – thời điểm phát hành, exp – thời điểm hết hạn) và chữ ký HMAC-SHA256 tính bằng khóa bí mật của server. Server không cần lưu phiên: mỗi request mang theo token, server kiểm tra chữ ký và thời hạn rồi tin vào claim bên trong. Ưu điểm là **stateless**, dễ chạy nhiều instance phía sau load balancer; nhược điểm là không thu hồi được token trước hạn nếu không có danh sách đen, vì vậy đề tài đặt thời hạn 24 giờ và để dành refresh token ở hướng phát triển.

BCrypt là hàm băm mật khẩu thích ứng: có muối (salt) ngẫu nhiên cho từng mật khẩu và tham số chi phí (cost) làm phép băm chậm có chủ đích, khiến việc dò mật khẩu bằng GPU trở nên tốn kém. Hệ thống dùng cost mặc định của Spring Security và chỉ lưu chuỗi băm (60 ký tự) – không bao giờ lưu hay trả về mật khẩu gốc.

2.8. Quản lý schema, kiểm thử và giám sát

- **Flyway** áp dụng các script SQL đánh số phiên bản (V1__init.sql) theo thứ tự khi ứng dụng khởi động và ghi lịch sử vào bảng flyway_schema_history; nhờ đó mọi môi trường có cùng schema và thay đổi được review như mã nguồn.
- **Testcontainers** khởi chạy PostgreSQL và SeaweedFS thật bằng Docker ngay trong bài test, loại bỏ tình trạng "chạy được trên máy tôi" do dùng mock khác với dịch vụ thật.
- **Micrometer + Prometheus** thu thập metric theo mô hình pull: Prometheus định kỳ gọi /actuator/prometheus; Grafana truy vấn bằng PromQL để vẽ biểu đồ (số request, độ trễ phân vị 95, heap JVM, số kết nối pool).
- **RFC 7807 Problem Details** chuẩn hóa thân phản hồi lỗi (type, title, status, detail) để client xử lý thống nhất.

2.9. Presigned URL so với tải qua máy chủ ứng dụng

Bảng 2.5. Hai cách phục vụ nội dung file

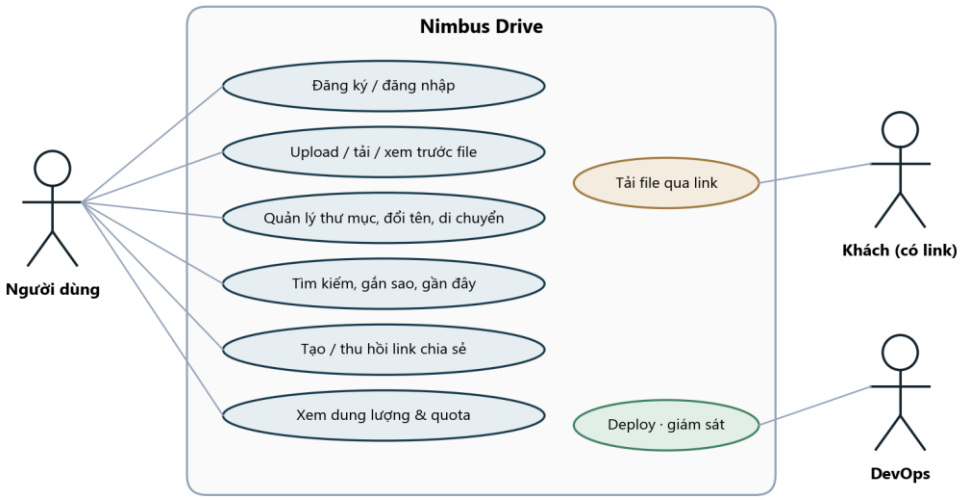
Tiêu chí	Tải qua máy chủ ứng dụng	Presigned URL tới S3
Đường đi dữ liệu	S3 → app → trình duyệt	S3 → trình duyệt (trực tiếp)
Tải cho app (CPU, băng thông)	Cao: app phải stream toàn bộ file	Gần như bằng 0
Kiểm soát quyền	Mỗi request đi qua JWT filter	Quyền được 'đóng gói' vào URL, hết hạn sau 15 phút
Yêu cầu hạ tầng	Chỉ cần mở cổng app	Trình duyệt phải truy cập được endpoint S3 (CORS)
Dùng trong đề tài	Tải xuống và xem trước trong	Menu "Link S3 tạm thời (15

	giao diện (kèm JWT)	phút)” và API /presign

Đề tài hỗ trợ cả hai: luồng đi qua ứng dụng đảm bảo mọi truy cập đều được xác thực và đếm lượt; presigned URL dành cho tải file lớn để giải phóng máy chủ. Khi chạy với LocalObjectStorage (dev), API presign trả 501 Not Implemented vì không có S3 phía sau (có test kiểm tra hành vi này).

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Yêu cầu chức năng



Hình 3.1. Sơ đồ use case tổng quát

Bảng 3.1. Đặc tả một số use case chính

Use case	Tiền điều kiện	Luồng chính	Luồng thay thế / lỗi
UC1. Upload file	Đã đăng nhập; thư mục đích thuộc về mình	Chọn/kéo file → hệ thống kiểm tra quota → ghi S3 → ghi metadata → hiện trong danh sách	Quá quota → 413; file rỗng → 400; storage lỗi → 502; DB lỗi → xóa object vừa ghi
UC2. Chia sẻ file	Là chủ sở hữu file	Chọn file → chọn thời hạn (1 giờ … 30 ngày hoặc vô hạn) → nhận link /share.html?t=<token>	File của người khác → 404; link quá hạn khi khách mở → 410
UC3. Khách tải file	Có link còn hiệu lực	Mở link → xem tên, kích thước, loại → bấm Tải xuống → tăng download_count	Link bị thu hồi → 404; file đã bị xóa → 404
UC4. Xóa thư mục	Là chủ sở hữu thư mục	Xác nhận → xóa đệ quy thư mục con và file → xóa object trên S3 sau khi commit	Thư mục của người khác → 404
UC5. Đăng nhập	Có tài khoản	Nhập tên + mật khẩu → nhận JWT → chuyển tới trang quản lý file	Sai 5 lần liên tiếp → khóa 5 phút (429); sai thông tin → 401

Bảng 3.2. Yêu cầu chức năng

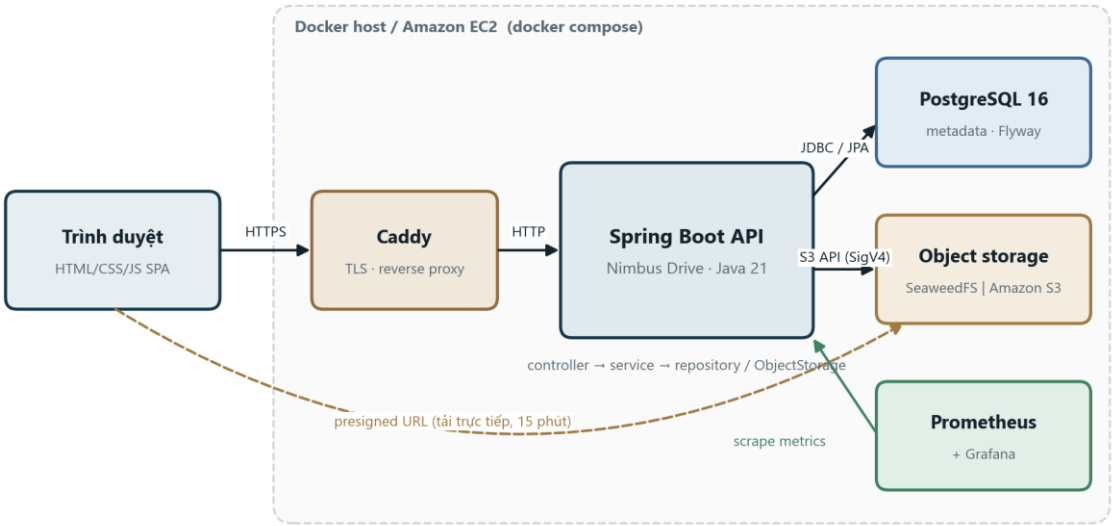
Mã	Chức năng	Mô tả
F01	Đăng ký / đăng nhập	Tên đăng nhập 3–50 ký tự [A-Za-z0-9._-], mật khẩu ≥ 6 ký tự; trả về JWT hạn 24 giờ
F02	Upload file	Kéo-thả hoặc chọn nhiều file, hiển thị tiến độ; giới hạn 100 MB/file, kiểm tra quota
F03	Tải xuống / xem	Xem ảnh, PDF, video, audio, văn bản/mã nguồn trên trình duyệt

	trước	
F04	Quản lý thư mục	Tạo, đổi tên, xóa đệ quy, thư mục lồng nhau, breadcrumb, di chuyển file
F05	Tổ chức	Đổi tên, gắn sao, mục Gần đây, tìm kiếm theo tên trên toàn kho
F06	Chia sẻ	Tạo link công khai có thời hạn, đếm lượt tải, thu hồi; trang tải cho khách
F07	Presigned URL	Sinh URL S3 ký sẵn 15 phút để tải trực tiếp từ object storage
F08	Dung lượng	Hiển thị dung lượng đã dùng / quota và phân bố theo loại file

3.2. Yêu cầu phi chức năng

- **Bảo mật:** mật khẩu BCrypt; JWT ký HS256; khóa tài khoản tạm thời sau 5 lần đăng nhập sai; người dùng chỉ truy cập tài nguyên của mình (trả 404 thay vì 403 để không lộ sự tồn tại); CSP chặn script lạ; không phục vụ inline HTML/SVG do người dùng tải lên.
- **Khả năng di chuyển (portability):** chạy với AWS S3 hoặc S3-compatible bất kỳ; cấu hình hoàn toàn qua biến môi trường (12-factor).
- **Khả năng quan sát:** health/liveness/readiness probe, metrics Prometheus, metric nghiệp vụ (số file, dung lượng upload).
- **Tin cậy:** migration CSDL có phiên bản (Flyway); không để object mồ côi khi lỗi; tự thử lại khi storage khởi động chậm; graceful shutdown.
- **Khả dụng giao diện:** responsive trên điện thoại, chế độ sáng/tối, thông báo lỗi tiếng Việt rõ ràng.

3.3. Kiến trúc tổng thể

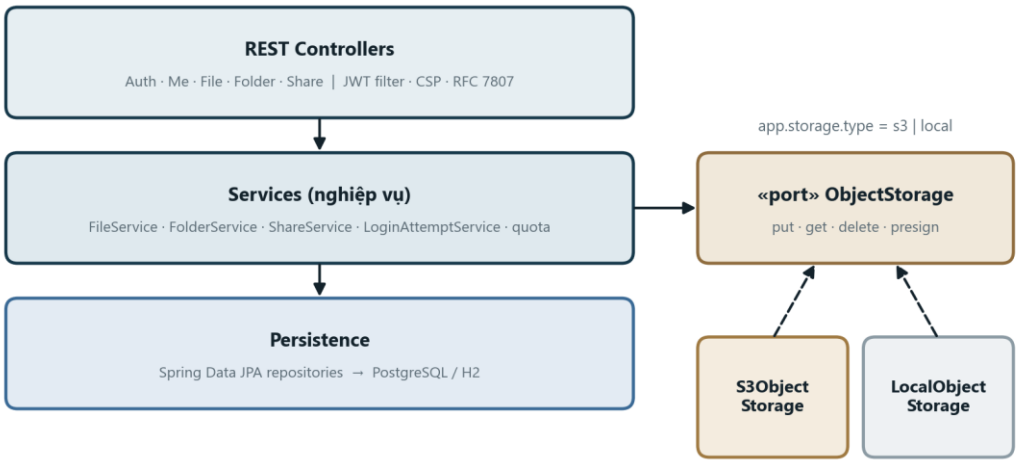


Hình 3.2. Kiến trúc tổng thể hệ thống Nimbus Drive

Hệ thống gồm các thành phần chạy trong container: **Caddy** làm reverse proxy và tự cấp chứng chỉ TLS (Let's Encrypt); **Spring Boot API** xử lý nghiệp vụ và phục vụ giao diện web tĩnh; **PostgreSQL** lưu người dùng, thư mục, metadata file và link chia sẻ; **object storage** lưu nội dung file. Khi chạy local/self-host, storage là SeaweedFS; khi chạy trên AWS là Amazon S3. Prometheus thu thập metrics và Grafana hiển thị dashboard (tùy chọn).

Việc tách metadata và nội dung là quyết định thiết kế quan trọng: CSDL quan hệ phù hợp cho truy vấn, phân quyền, tìm kiếm; object storage phù hợp cho dữ liệu nhị phân lớn, rẻ và bền. Thư mục là **thư mục ảo** chỉ tồn tại trong DB – object trên S3 được đặt key phẳng dạng owner/uuid, nên đổi tên hay di chuyển thư mục không cần sao chép dữ liệu trên storage.

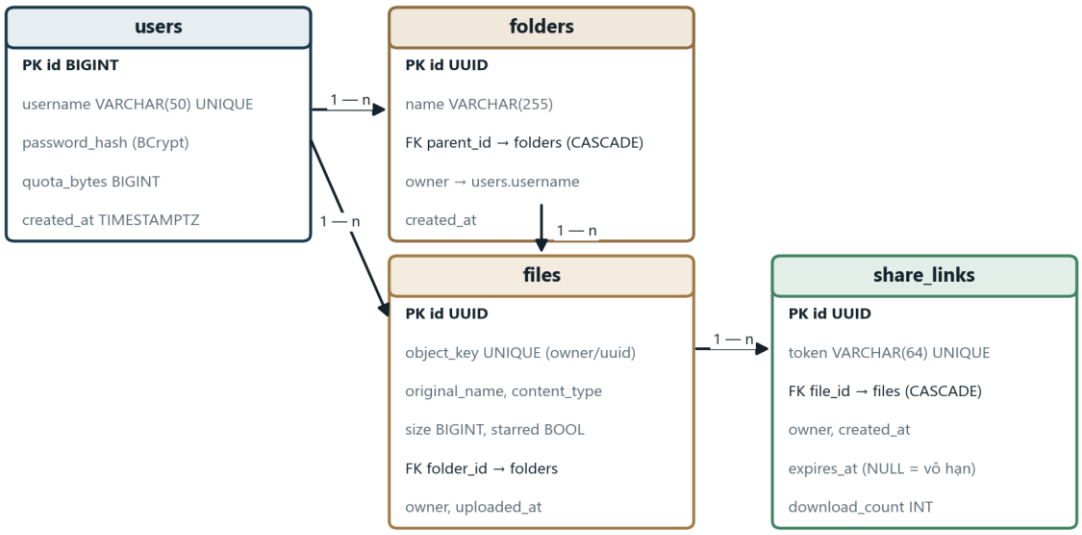
3.4. Kiến trúc phần mềm



Hình 3.3. Kiến trúc phân lớp và mẫu Ports & Adapters cho tầng lưu trữ

Mã nguồn chia lớp: Controller (REST, validate đầu vào) → Service (nghiệp vụ, kiểm tra quyền sở hữu, quota, transaction) → Repository (Spring Data JPA). Tầng lưu trữ được trừu tượng hóa bằng interface **ObjectStorage** (port) với hai adapter: **S3ObjectStorage** (AWS SDK v2) và **LocalObjectStorage** (ổ đĩa, dùng cho phát triển/kiểm thử). Adapter được chọn qua thuộc tính app.storage.type nhờ @ConditionalOnProperty.

3.5. Thiết kế cơ sở dữ liệu



Hình 3.4. Lược đồ quan hệ thực thể (ERD)

Bảng 3.3. Mô tả các bảng

Bảng	Mục đích	Ràng buộc đáng chú ý
users	Tài khoản, mật khẩu băm, quota	username UNIQUE
folders	Cây thư mục ảo của từng người dùng	parent_id tự tham chiếu, ON DELETE CASCADE; index (owner, parent_id)
files	Metadata file, ánh xạ tới object key	object_key UNIQUE; index (owner, folder_id)
share_links	Link chia sẻ công khai	token UNIQUE (192 bit ngẫu nhiên); file_id ON DELETE CASCADE

Schema được quản lý bằng **Flyway** (V1__init.sql); Hibernate đặt ddl-auto = none để tránh thay đổi schema ngầm định ở production. Cùng một script migration chạy được trên PostgreSQL (production) và H2 chế độ PostgreSQL (dev/test).

Bảng 3.4. Từ điển dữ liệu – bảng users và folders

Cột	Kiểu	Ràng buộc	Ý nghĩa
users.id	BIGINT identity	PK	Khóa chính tự tăng
users.username	VARCHAR(50)	NOT NULL, UNIQUE	Tên đăng nhập, dùng làm định danh owner ở các bảng khác
users.password_hash	VARCHAR(100)	NOT NULL	Chuỗi băm BCrypt (60 ký tự)
users.quota_bytes	BIGINT	NOT NULL	Hạn mức dung lượng, mặc định 1.073.741.824 byte (1 GB)
users.created_at	TIMESTAMPTZ	NOT NULL	Thời điểm tạo tài khoản (“thành viên từ”)
folders.id	UUID	PK	Sinh ở phía ứng dụng để

			không lộ thứ tự tạo
folders.name	VARCHAR(255)	NOT NULL	Tên đã được làm sạch (FileNames.clean)
folders.parent_id	UUID	FK folders.id, ON DELETE CASCADE	NULL = thư mục gốc; xóa cha thì xóa luôn con
folders.owner	VARCHAR(50)	NOT NULL, index (owner, parent_id)	Chủ sở hữu; mọi truy vấn đều lọc theo cột này

Bảng 3.5. Từ điển dữ liệu – bảng files và share_links

Cột	Kiểu	Ràng buộc	Ý nghĩa
files.object_key	VARCHAR(255)	NOT NULL, UNIQUE	Khóa object trên S3: owner/<uuid>
files.original_name	VARCHAR(255)	NOT NULL	Tên hiển thị, đã loại ký tự nguy hiểm
files.content_type	VARCHAR(255)	NULL được	MIME type; suy từ phần mở rộng nếu client gửi octet-stream
files.size	BIGINT	NOT NULL	Số byte; dùng cộng dồn để kiểm tra quota
files.folder_id	UUID	FK folders.id	NULL = ở thư mục gốc; xóa thư mục xử lý ở tầng service để dọn object
files.starred	BOOLEAN	DEFAULT FALSE	Đánh dấu sao
share_links.token	VARCHAR(64)	NOT NULL, UNIQUE	24 byte ngẫu nhiên mã hóa Base64URL (32 ký tự)
share_links.file_id	UUID	FK files.id, ON DELETE CASCADE	File được chia sẻ; xóa file thì link biến mất
share_links.expires_at	TIMESTAMPTZ	NULL được	NULL = không hết hạn
share_links.download_count	INT	DEFAULT 0	Số lượt khách đã tải

3.6. Thiết kế API

Bảng 3.6. Các endpoint REST chính

Method	Endpoint	Mô tả
POST	/api/auth/register, /api/auth/login	Đăng ký / đăng nhập, trả JWT
GET	/api/me	Hồ sơ, dung lượng, thống kê theo loại
GET	/api/files?folderId=&q=&view=	Liệt kê theo thư mục, tìm kiếm, recent/starred
POST	/api/files (multipart)	Upload file vào thư mục
GET	/api/files/{id}/download?inline=	Tải hoặc xem trước
GET	/api/files/{id}/presign	Presigned URL S3 (15 phút)

PATCH / POST / PUT / DELETE	/api/files/{id}, /move, /star	Đổi tên, di chuyển, gắn sao, xóa
GET / POST / PATCH / DELETE	/api/folders, /api/folders/{id}/path	Quản lý thư mục, breadcrumb
POST	/api/files/{id}/shares	Tạo link chia sẻ có thời hạn
GET / DELETE	/api/shares, /api/shares/{id}	Danh sách / thu hồi link
GET	/api/public/shares/{token}[/download]	Công khai: thông tin và tải file

Lỗi được trả về theo chuẩn **RFC 7807 Problem Details** (application/problem+json) với thông điệp tiếng Việt, ví dụ 413 "Vượt quá dung lượng cho phép", 429 khi đăng nhập sai quá nhiều, 410 khi link chia sẻ hết hạn, 502 khi không kết nối được storage.

Bảng 3.7. Bảng mã trạng thái HTTP của API

Mã	**Khi nào**	**Ví dụ**
200 / 201 / 204	Thành công / tạo mới / xóa thành công	POST /api/files → 201; DELETE /api/files/{id} → 204
400	Dữ liệu không hợp lệ (Bean Validation), file rỗng	Mật khẩu ngắn hơn 6 ký tự; upload file 0 byte
401	Thiếu/sai/hết hạn JWT hoặc sai mật khẩu	Gọi /api/me không có header Authorization
404	Không tồn tại hoặc không thuộc về người dùng	Truy cập file của người khác; link đã thu hồi
409	Trùng tài khoản	Đăng ký username đã tồn tại
410	Link chia sẻ hết hạn	Khách mở link sau expires_at
413	Vượt quota hoặc quá kích thước tối đa	Upload khi đã dùng hết 1 GB; file > 100 MB
429	Đăng nhập sai quá nhiều lần	Lần sai thứ 5 → khóa 5 phút
501	Backend lưu trữ không hỗ trợ tính năng	Presigned URL khi chạy LocalObjectStorage
502	Không kết nối được object storage	SeaweedFS/S3 chưa sẵn sàng hoặc sai credential

Ví dụ gọi API bằng curl (thân phản hồi rút gọn):

```
# 1. Đăng ký và lấy token
curl -s -X POST http://localhost:8080/api/auth/register -H "Content-Type:
application/json" \
     -d '{"username":"alice","password":"secret123"}'
# -> 201 {"token":"eyJhbGciOi...","username":"alice","expiresInMs":86400000}

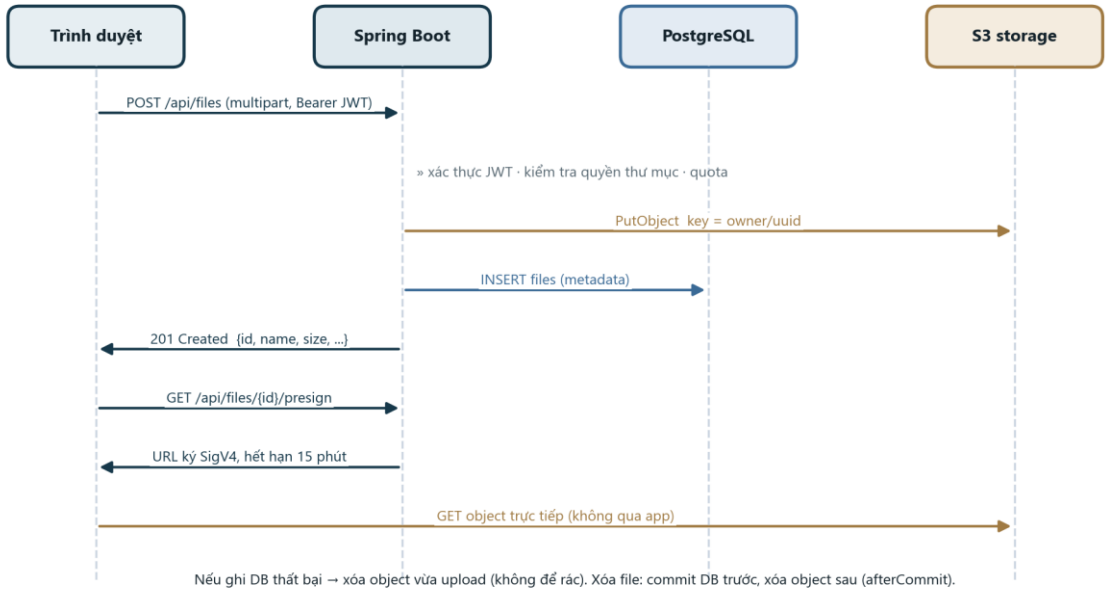
# 2. Upload file vào thư mục gốc
curl -s -X POST http://localhost:8080/api/files -H "Authorization: Bearer $TOKEN" -F
"file=@report.pdf"
# -> 201
{"id":"3f1c...","name":"report.pdf","contentType":"application/pdf","size":182340,"sta
rred":false,...}

# 3. Tạo link chia sẻ 7 ngày
```

```
curl -s -X POST http://localhost:8080/api/files/3f1c.../shares -H "Authorization:
Bearer $TOKEN" \
     -H "Content-Type: application/json" -d '{"expiresInHours":168}'

# 4. Khách tải không cần đăng nhập
curl -OJ http://localhost:8080/api/public/shares/<token>/download
```

3.7. Luồng xử lý upload và tải qua presigned URL



Hình 3.5. Sơ đồ tuần tự upload file và tải trực tiếp bằng presigned URL

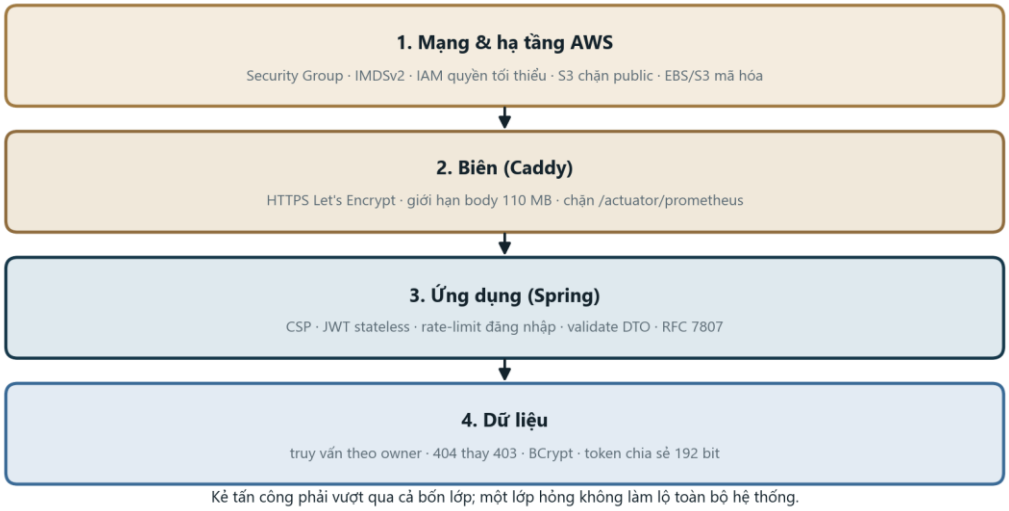
Khi upload, service kiểm tra JWT, quyền với thư mục đích và quota, sau đó ghi object lên S3 trước rồi mới ghi metadata. Nếu ghi DB thất bại, object vừa ghi bị xóa để không để lại dữ liệu mồ côi. Khi xóa, metadata được xóa trong transaction, còn object chỉ bị xóa **sau khi commit** thành công (TransactionSynchronization.afterCommit) – tránh trường hợp mất dữ liệu thật nhưng metadata vẫn còn.

3.8. Thiết kế bảo mật

Bảng 3.8. Các biện pháp bảo mật

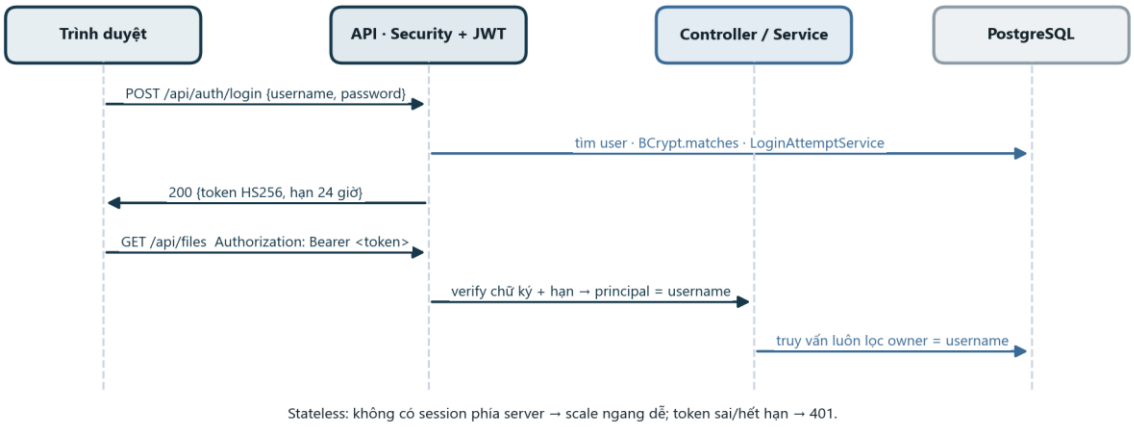
Mối đe dọa	Biện pháp
Lộ mật khẩu	BCrypt (cost 10); không bao giờ trả về hash
Dò mật khẩu (brute force)	LoginAttemptService khóa 5 phút sau 5 lần sai liên tiếp (HTTP 429)
Truy cập trái phép (IDOR)	Mọi truy vấn lọc theo owner; tài nguyên của người khác → 404
XSS qua file tải lên	Chỉ hiển thị inline ảnh raster/PDF/text/video/audio; HTML, SVG… ép tải về octet-stream; header CSP sandbox trên response file
XSS trên giao diện	CSP script-src 'self', không inline script/handler; escape mọi dữ liệu khi render

Path traversal	Làm sạch tên file (bỏ ../, ký tự điều khiển); LocalObjectStorage kiểm tra key nằm trong thư mục gốc
Đoán link chia sẻ	Token 24 byte SecureRandom (192 bit), Base64URL; có thời hạn và thu hồi được
Lộ credential S3 trên AWS	Dùng IAM instance profile, không lưu access key; IMDSv2 bắt buộc; policy chỉ Get/Put/Delete trên 1 bucket
Lộ dữ liệu trên S3	Block Public Access, mã hóa SSE-S3, versioning, BucketOwnerEnforced
Nghe lén	HTTPS qua Caddy (Let's Encrypt), app chỉ lắng nghe 127.0.0.1 trên EC2



Hình 3.6. Mô hình phòng thủ nhiều lớp (defense in depth)

3.8.1. Luồng xác thực JWT

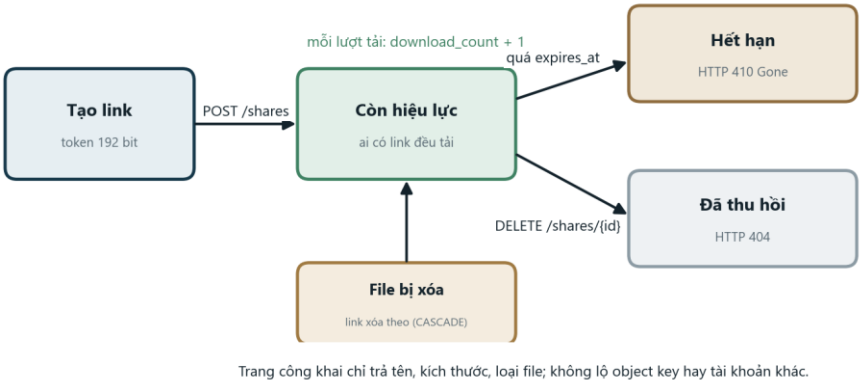


Hình 3.7. Luồng đăng nhập và gọi API bằng Bearer token

Khi đăng nhập, LoginAttemptService kiểm tra tài khoản có đang bị khóa không; nếu mật khẩu đúng, bộ đếm sai được xóa và JwtService ký token HMAC-SHA (HS256 trở lên, thư viện JJWT tự chọn theo độ dài khóa) với khóa tối thiểu 32 ký tự lấy từ biến môi trường JWT_SECRET. Với mỗi request tiếp theo, JwtAuthFilter đọc header Authorization, xác minh chữ ký và thời hạn rồi đặt tên người dùng làm

principal; mọi service nhận tên này làm tham số owner nên không thể truy vấn dữ liệu của người khác ngay cả khi client gửi sai id.

3.8.2. Vòng đời link chia sẻ



Hình 3.8. Các trạng thái của link chia sẻ công khai

Link công khai là ngoại lệ duy nhất của nguyên tắc "mọi thứ yêu cầu đăng nhập", nên được thiết kế chặt: token 192 bit không thể đoán (2^192 khả năng), có thời hạn tùy chọn, thu hồi được bất cứ lúc nào, và trang công khai chỉ trả thông tin tối thiểu (tên, kích thước, loại, người chia sẻ). Mỗi lượt tải qua link cũng đi qua cùng cơ chế chống XSS như tải thông thường.

3.9. Các quyết định thiết kế quan trọng

Bảng 3.9. Bảng quyết định kiến trúc (ADR rút gọn)

Vấn đề	Phương án đã chọn	Lý do	Đánh đổi
Nơi lưu nội dung file	Object storage chuẩn S3	Rẻ, bền, mở rộng; có presigned URL	Độ trễ cao hơn ổ đĩa cục bộ
Cấu trúc thư mục	Thư mục ảo trong PostgreSQL	Đổi tên/di chuyển không copy dữ liệu; truy vấn dễ	Cần xử lý xóa đệ quy ở tầng service
Đặt key object	owner/<uuid>, không dùng tên file	Tránh trùng, tránh ký tự lạ, không lộ tên	Không duyệt được bucket bằng mắt thường
Xác thực	JWT stateless	Không cần session store; dễ scale ngang	Khó thu hồi trước hạn
Truy cập tài nguyên người khác	Trả 404 thay vì 403	Không lộ sự tồn tại của tài nguyên	Khó phân biệt lỗi khi debug
Quản lý schema	Flyway, ddl-auto = none	Schema có phiên bản, an toàn ở production	Phải viết SQL thủ công
Frontend	JavaScript thuần, không bước build	CSP chặt, dễ triển khai, ít phụ thuộc	Mã giao diện dài hơn so với dùng framework
Hạ tầng AWS	1 EC2 + S3 qua Terraform	Đơn giản, chi phí thấp, đủ chứng minh khái niệm	Không có Multi-AZ, DB là điểm lỗi đơn
Thứ tự ghi	S3 trước, DB sau;	Không bao giờ có	Có thể sót object mồ

	xóa DB trước, S3 sau	metadata trỏ tới object không tồn tại	côi nếu app sập giữa chừng

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

4.1. Cấu trúc mã nguồn

```
cloud-file-storage/
├── src/main/java/com/cloudfs/
│   ├── config/        StorageConfig (S3Client, S3Presigner), OpenApiConfig
│   ├── controller/    Auth, Me, File, Folder, Share
│   ├── service/       FileService, FolderService, ShareService, LoginAttemptService
│   ├── storage/       ObjectStorage, S3ObjectStorage, LocalObjectStorage
│   ├── security/      JwtService, JwtAuthFilter, SecurityConfig
│   └── web/           ApiExceptionHandler (RFC 7807)
├── src/main/resources/
│   ├── db/migration/V1__init.sql
│   └── static/        index.html, app.html, share.html, css/, js/
├── src/test/          ApiFlowTest, UnitTests, *IT (Testcontainers)
├── deploy/            seaweedfs/, caddy/, monitoring/, terraform/
├── Dockerfile, docker-compose.yml, docker-compose.aws.yml
└── .github/workflows/ci-cd.yml
```

4.2. Tầng lưu trữ tương thích S3

StorageConfig tạo S3Client và S3Presigner từ cấu hình. Khi S3_ENDPOINT rỗng, SDK dùng endpoint AWS thật; khi có giá trị, ứng dụng kết nối tới storage tương thích (SeaweedFS) với path-style addressing. Nếu không khai báo access key, ứng dụng dùng **DefaultCredentialsProvider** – trên EC2 tự lấy credential tạm thời từ IAM role. Presigner có thể dùng endpoint công khai riêng (S3_PUBLIC_ENDPOINT) vì URL ký sẵn phải trỏ tới địa chỉ mà trình duyệt truy cập được.

```
public interface ObjectStorage {
    void put(String key, InputStream content, long size, String contentType) throws
IOException;
    InputStream get(String key) throws IOException;
    void delete(String key);
    Optional<URL> presignDownload(String key, String fileName, Duration ttl);
    String backendName();
}

// S3ObjectStorage.presignDownload
var request = GetObjectRequest.builder().bucket(bucket).key(key)

.responseContentDisposition(ContentDispositions.attachment(fileName)).build();
return Optional.of(presigner.presignGetObject(p -> p.signatureDuration(ttl)
        .getObjectRequest(request)).url());
```

Khi khởi động, S3ObjectStorage.ensureBucket() kiểm tra (HeadBucket) và tạo bucket nếu chưa có, thử lại tối đa 10 lần cách nhau 3 giây để chịu được việc storage khởi động chậm hơn ứng dụng trong Docker Compose.

4.2.1. Thứ tự ghi dữ liệu khi upload và xóa

Do dữ liệu nằm ở hai nơi (S3 và PostgreSQL) không có giao dịch phân tán, mã nguồn phải chọn thứ tự thao tác sao cho **lỗi ở bất kỳ bước nào cũng không để lại**

metadata trỏ tới object không tồn tại. Khi upload, object được ghi trước; chỉ khi ghi metadata thất bại mới phải dọn dẹp. Khi xóa, metadata bị xóa trong transaction và object chỉ bị xóa sau khi transaction commit.

```
// FileService.upload – ghi S3 trước, DB sau, lỗi DB thì xóa object
storage.put(key, in, file.getSize(), type);
try {
    FileMeta meta = files.save(new FileMeta(id, key, name, type, file.getSize(),
owner, folderId));
    uploads.increment();                        // metric nghiệp vụ
    uploadBytes.record(file.getSize());
    return meta;
} catch (RuntimeException e) {
    storage.delete(key);                        // không để object mồ côi
    throw e;
}

// FileService.deleteAll – chỉ xóa object SAU KHI commit DB thành công
TransactionSynchronizationManager.registerSynchronization(new
TransactionSynchronization() {
    @Override public void afterCommit() { removeObjects.run(); }
});
```

Trường hợp còn lại duy nhất là ứng dụng bị tắt đột ngột giữa hai bước, khi đó có thể còn một object không có metadata (rác nhưng không gây lỗi hay lộ dữ liệu). Hướng phát triển đề xuất một job định kỳ đối chiếu danh sách object với bảng files để dọn rác.

4.2.2. Kiểm soát quyền sở hữu, quota và tên file

- **Quyền sở hữu:** mọi phương thức service đều nhận tham số owner lấy từ JWT và gọi findOwned; kết quả không thuộc owner bị coi như không tồn tại (404).
- **Quota:** trước khi ghi, hệ thống cộng dồn dung lượng đã dùng (SUM size theo owner) và từ chối nếu tổng vượt quota_bytes.
- **Tên file:** FileNames.clean cắt phần đường dẫn (cả / và \), loại ký tự điều khiển, từ chối "." và "..", giới hạn 255 ký tự; tên gốc chỉ để hiển thị, không bao giờ dùng làm đường dẫn trên storage.
- **Kiểu nội dung:** nếu client gửi application/octet-stream, hệ thống suy ra MIME từ phần mở rộng (MediaTypeFactory) để xem trước đúng.

4.3. Giao diện người dùng

Giao diện là SPA viết bằng JavaScript thuần, không cần bước build, được Spring Boot phục vụ dưới dạng tài nguyên tĩnh. Các thành phần chính: trang đăng nhập/đăng ký, trang quản lý file (sidebar điều hướng, thanh tìm kiếm, breadcrumb, lưới/danh sách, menu ngữ cảnh, hộp thoại xem trước, chia sẻ, di chuyển), bảng tiến độ upload và trang tải công khai. Upload dùng XMLHttpRequest để có sự kiện tiến độ; ảnh thu nhỏ và bản xem trước được tải bằng fetch kèm JWT rồi hiển thị qua blob URL.

Bảng 4.1. Các tệp giao diện và trách nhiệm

Tệp	Dòng	Trách nhiệm
index.html + js/auth.js	71	Đăng nhập/đăng ký, lưu JWT, chuyển hướng, hiển thị lỗi từ server
app.html + js/app.js	745	Trang chính: gọi API, vẽ lưới/danh sách, upload có tiến độ, kéo-thả, menu ngữ cảnh, xem trước, chia sẻ, phím tắt
js/common.js	228	Hàm dùng chung: gọi API kèm JWT, định dạng dung lượng/ngày, icon, toast, hộp thoại, chuyển giao diện sáng/tối
share.html + js/share.js	45	Trang công khai cho khách: đọc token trên URL, hiển thị thông tin file và nút tải
css/style.css	347	Hệ thống thiết kế bằng biến CSS (màu, bo góc, bóng), hai bộ màu sáng/tối, bố cục responsive

Giao diện được thiết kế lại với bảng màu **mực và sâm panh** (ink & champagne), tiêu đề dùng phông có chân để tạo cảm giác biên tập, và nút chuyển chế độ sáng/tối ở thanh trên. Toàn bộ màu sắc là biến CSS (custom property) nên chế độ tối chỉ cần ghi đè các biến, không nhân đôi stylesheet. Vì CSP chỉ cho phép script từ cùng origin, không có script hay thuộc tính onclick nhúng trong HTML; mọi sự kiện được gắn bằng addEventListener và dữ liệu hiển thị luôn được escape trước khi đưa vào DOM.

4.4. Container hóa

4.4.1. Dockerfile nhiều giai đoạn

```
FROM maven:3.9-eclipse-temurin-21 AS build
RUN --mount=type=cache,target=/root/.m2 mvn -B -q dependency:go-offline
RUN --mount=type=cache,target=/root/.m2 mvn -B -q package -DskipTests \
    && java -Djarmode=tools -jar target/*.jar extract --layers --launcher --
destination extracted

FROM eclipse-temurin:21-jre
RUN useradd --system --uid 10001 appuser
COPY --from=build /workspace/extracted/dependencies/ ./
COPY --from=build /workspace/extracted/application/ ./
USER appuser
HEALTHCHECK CMD curl -fs http://localhost:8080/actuator/health/liveness || exit 1
ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS
org.springframework.boot.loader.launch.JarLauncher"]
```

- Giai đoạn build dùng JDK + Maven; image chạy chỉ chứa JRE → nhỏ gọn, ít bề mặt tấn công.
- Jar được tách **layer** (dependencies, loader, application): khi chỉ sửa mã, Docker chỉ build lại layer application.
- Chạy bằng user thường (UID 10001), có HEALTHCHECK, JVM giới hạn theo bộ nhớ container (MaxRAMPercentage=75).

4.4.2. Docker Compose

Bảng 4.2. Các service trong docker-compose.yml

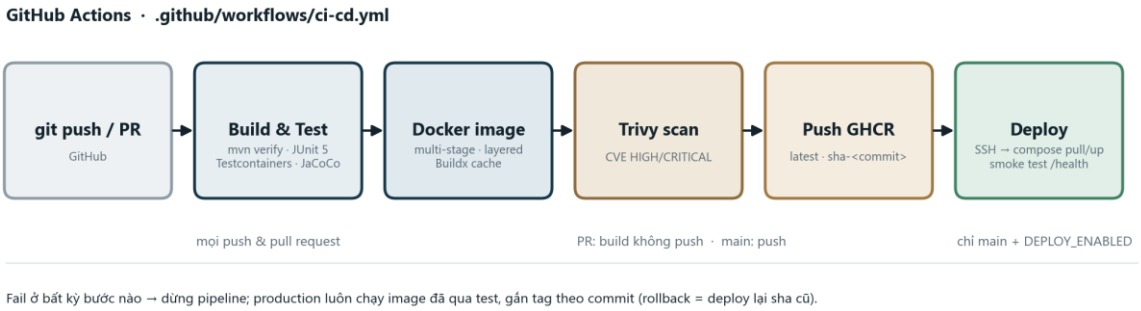
Service	Image	Vai trò
postgres	postgres:16-alpine	CSDL metadata, volume pgdata, healthcheck pg_isready
seaweedfs	chrislusf/seaweedfs:4.52	S3 server tự host, cổng 8333; danh tính S3 sinh từ biến môi trường
app	nimbus-drive (build từ Dockerfile)	Spring Boot; chờ postgres và seaweedfs healthy
prometheus, grafana	profile monitoring	Thu thập và hiển thị metrics, dashboard được provision sẵn
caddy	profile proxy	Reverse proxy HTTPS tự động, chặn public /actuator/prometheus

File docker-compose.aws.yml ghi đè cho môi trường AWS: tắt SeaweedFS, để trống endpoint và access key (dùng Amazon S3 thật qua IAM role), và chỉ mở cổng app trên 127.0.0.1 để mọi truy cập bên ngoài phải đi qua Caddy.

Bảng 4.3. Các biến môi trường cấu hình (nguyên tắc 12-factor)

Biến	Mặc định	Ý nghĩa
DB_URL, DB_USER, DB_PASSWORD	localhost / cloudfs	Kết nối PostgreSQL; mật khẩu sinh ngẫu nhiên trên AWS
STORAGE_TYPE	s3	s3 (AWS S3, SeaweedFS, MinIO…) hoặc local (thư mục, chỉ để phát triển)
S3_ENDPOINT	http://localhost:8333	Để trống = Amazon S3 thật; có giá trị = S3-compatible tự host
S3_PUBLIC_ENDPOINT	(= S3_ENDPOINT)	Địa chỉ mà trình duyệt truy cập được, dùng ký presigned URL
S3_REGION, S3_BUCKET	us-east-1 / cloudfs	Vùng và tên bucket
S3_ACCESS_KEY, S3_SECRET_KEY	cloudfs	Để trống trên AWS → dùng IAM instance profile
S3_PATH_STYLE	true	true cho SeaweedFS/MinIO; false cho Amazon S3
JWT_SECRET, JWT_EXPIRATION_MS	đổi khi deploy / 86400000	Khóa ký (≥ 32 ký tự) và thời hạn token (24 giờ)
DEFAULT_QUOTA_BYTES, MAX_FILE_SIZE	1 GB / 100 MB	Hạn mức mỗi người dùng và kích thước tối đa mỗi file
JAVA_OPTS	MaxRAMPercentage=75, G1GC	Tham số JVM, tự co giãn theo giới hạn bộ nhớ container (768 MB)
DOMAIN, APP_IMAGE, APP_PORT	localhost / nimbus-drive:local / 8080	Tên miền cho Caddy, image cần chạy, cổng công bố

4.5. Pipeline CI/CD với GitHub Actions



Hình 4.1. Pipeline CI/CD tự động

Bảng 4.4. Các job trong pipeline

Job	Kích hoạt	Nội dung
test	Mọi push và pull request	mvn verify: unit test, API test, integration test bằng Testcontainers (PostgreSQL, SeaweedFS); lưu báo cáo test và độ phủ JaCoCo
image	Sau test	Docker Buildx với cache GHA; PR chỉ build kiểm tra, main thì push GHCR tag latest + sha-<commit>; quét Trivy lỗ hổng HIGH/CRITICAL
deploy	Push main, DEPLOY_ENABLED=true	SCP file compose lên server, docker compose pull/up với image đúng commit, smoke test /actuator/health trong 90 giây, in log nếu thất bại

Image production được gắn tag theo commit nên có thể truy vết và **rollback** bằng cách triển khai lại tag cũ. Job deploy gắn với GitHub Environment "production", cho phép bật duyệt thủ công (required reviewers) nếu cần.

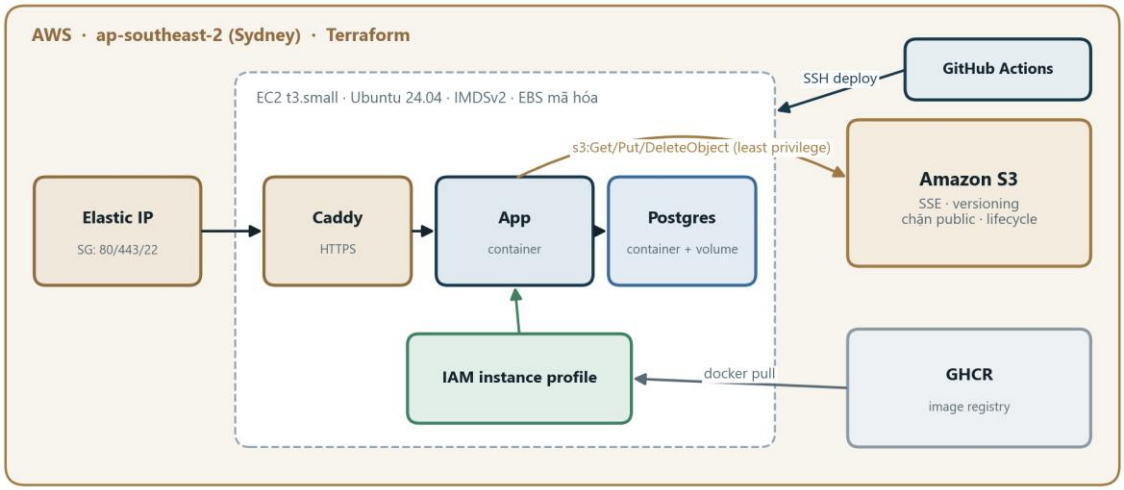
Bảng 4.5. Cấu hình GitHub cần thiết cho bước deploy

Loại	Tên	Giá trị / mục đích
Variable	DEPLOY_ENABLED	true để bật job deploy; false để chỉ build, test, đẩy image (không đụng tới server)
Variable	APP_URL	Địa chỉ hiển thị trong Environment production, ví dụ http://<Elastic IP>
Secret	DEPLOY_HOST	Elastic IP của EC2 (output public_ip của Terraform)
Secret	DEPLOY_USER	ubuntu (người dùng mặc định của AMI Ubuntu)
Secret	DEPLOY_SSH_KEY	Khóa riêng của key pair EC2; không bao giờ commit vào repo
Tự động	GITHUB_TOKEN	Cấp quyền packages: write cho job image và packages: read cho job deploy để server kéo image từ GHCR

Pipeline áp dụng một số thực hành an toàn: nhóm concurrency hủy các lần chạy cũ khi có push mới trên cùng nhánh; hành động Trivy được ghim theo mã commit (SHA) thay vì thẻ phiên bản để tránh bị thay đổi ngầm; báo cáo test và độ phủ được

lưu 14 ngày; và một bước riêng kiểm tra failsafe-summary.xml có skipped = 0 để pipeline không báo xanh giả khi integration test bị bỏ qua.

4.6. Hạ tầng AWS bằng Terraform



Hình 4.2. Kiến trúc triển khai trên AWS

Bảng 4.6. Tài nguyên Terraform tạo ra

Tài nguyên	Cấu hình
aws_s3_bucket + cấu hình	Block Public Access, SSE-AES256, versioning, lifecycle (xóa phiên bản cũ sau 30 ngày, hủy multipart dở sau 7 ngày), CORS cho presigned GET
aws_iam_role / instance_profile	Chỉ s3:ListBucket trên bucket và s3:Get/Put/DeleteObject trên bucket/*; kèm AmazonSSMManagedInstanceCore
aws_security_group	Mở 80, 443 công khai; 22 theo danh sách CIDR
aws_instance	Ubuntu 24.04, t3.small, IMDSv2 bắt buộc, EBS gp3 mã hóa; user_data cài Docker và sinh file .env (mật khẩu DB, JWT secret ngẫu nhiên)
aws_eip	Địa chỉ IP tĩnh để trỏ DNS và làm DEPLOY_HOST

```
cd deploy/terraform
cp terraform.tfvars.example terraform.tfvars   # key_name, app_image, domain
terraform init
terraform plan
terraform apply        # output: public_ip, bucket, app_url
```

Bảng 4.7. Biến Terraform chính

Biến	Mặc định	Ý nghĩa
region	ap-southeast-1 (đặt ap-southeast-2 khi triển khai)	Vùng AWS
instance_type, disk_gb	t3.small, 20	Cấu hình máy chủ và dung lượng ổ EBS gp3
key_name	(bắt buộc)	Tên EC2 key pair để SSH và để GitHub Actions deploy

ssh_cidrs	0.0.0.0/0	Dải IP được SSH; nên thu hẹp lại, GitHub-hosted runner cần dải rộng
domain	(trống)	Có tên miền thì Caddy tự xin chứng chỉ HTTPS; trống thì chỉ HTTP qua IP
app_image	(bắt buộc)	Image trên GHCR, ví dụ ghcr.io/<owner>/<repo>:latest
force_destroy_bucket	false	true cho phép terraform destroy xóa cả bucket còn dữ liệu (chỉ dùng khi demo)

Các giá trị nhạy cảm (mật khẩu PostgreSQL, JWT_SECRET) được Terraform sinh bằng random_password và truyền vào user_data; chúng nằm trong file .env phân quyền 600 trên EC2 và trong state của Terraform. Vì vậy file terraform.tfstate không được commit và nên đặt trên backend từ xa (S3 + DynamoDB lock) khi làm việc nhóm.

4.7. Giám sát

Spring Boot Actuator cung cấp /actuator/health (kèm liveness/readiness), /actuator/prometheus. Ngoài các metric có sẵn (HTTP request, độ trễ p95, JVM heap, kết nối Hikari), ứng dụng đăng ký metric nghiệp vụ nimbus.files.uploaded và nimbus.files.upload.bytes. Dashboard Grafana "Nimbus Drive – Overview" được provision tự động khi bật profile monitoring.

Bảng 4.8. Các biểu đồ trên dashboard Grafana

Biểu đồ	**Nguồn metric**	**Dùng để**
Uptime	up của job nimbus-drive	Biết ứng dụng còn sống để Prometheus scrape được không
HTTP requests / s	http_server_requests_seconds_count	Lưu lượng truy cập theo thời gian
p95 latency	http_server_requests_seconds_bucket (histogram)	Độ trễ phân vị 95, phát hiện chậm bất thường
5xx (5m)	http_server_requests với status 5xx	Tỷ lệ lỗi phía server trong 5 phút
Files uploaded / Bytes uploaded	nimbus_files_uploaded, nimbus_files_upload_bytes	Chỉ số nghiệp vụ: số file và dung lượng tải lên
JVM heap used	jvm_memory_used_bytes	Theo dõi bộ nhớ, phát hiện rò rỉ
DB connections (Hikari)	hikaricp_connections_active	Số kết nối CSDL đang dùng, phát hiện nghẽn pool

Prometheus scrape mỗi 15 giây qua /actuator/prometheus. Endpoint này được Caddy trả 404 với truy cập từ Internet nên chỉ truy cập được trong mạng Docker. Health check của Docker dùng /actuator/health/liveness, còn smoke test của pipeline dùng /actuator/health.

4.8. Quy trình triển khai, nâng cấp và rollback

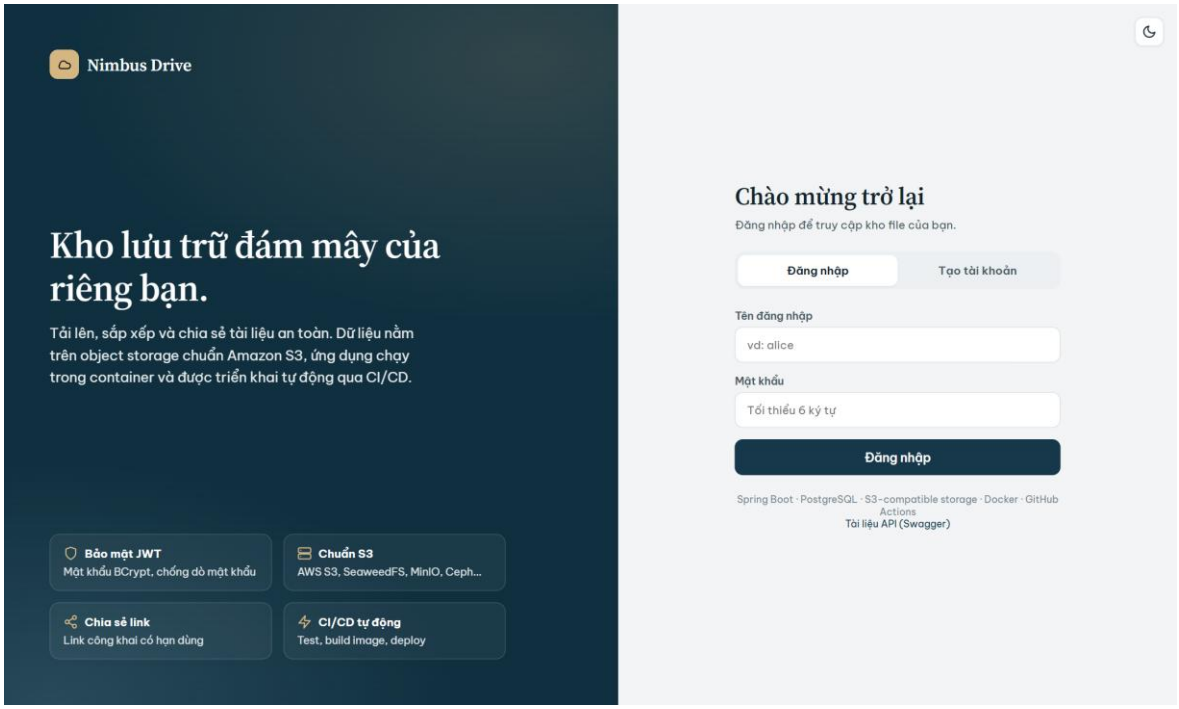
Bảng 4.9. Quy trình triển khai từ đầu tới cuối

Bước	Việc cần làm	Lệnh / nơi thực hiện
1	Đăng nhập AWS và chuẩn bị key pair EC2	aws login (hoặc aws configure); tạo key pair trong EC2 console
2	Khai báo biến Terraform	cp terraform.tfvars.example terraform.tfvars, điền key_name, app_image, region
3	Tạo hạ tầng (S3, IAM, SG, EC2, EIP)	terraform init && terraform apply
4	Cấu hình GitHub: secrets, DEPLOY_ENABLED=true	Repository → Settings → Secrets and variables
5	Đẩy mã lên main để pipeline test, build, deploy	git push origin main (hoặc Run workflow)
6	Kiểm tra	curl http://<Elastic IP>/actuator/health → UP; mở trang web, đăng ký và upload thử

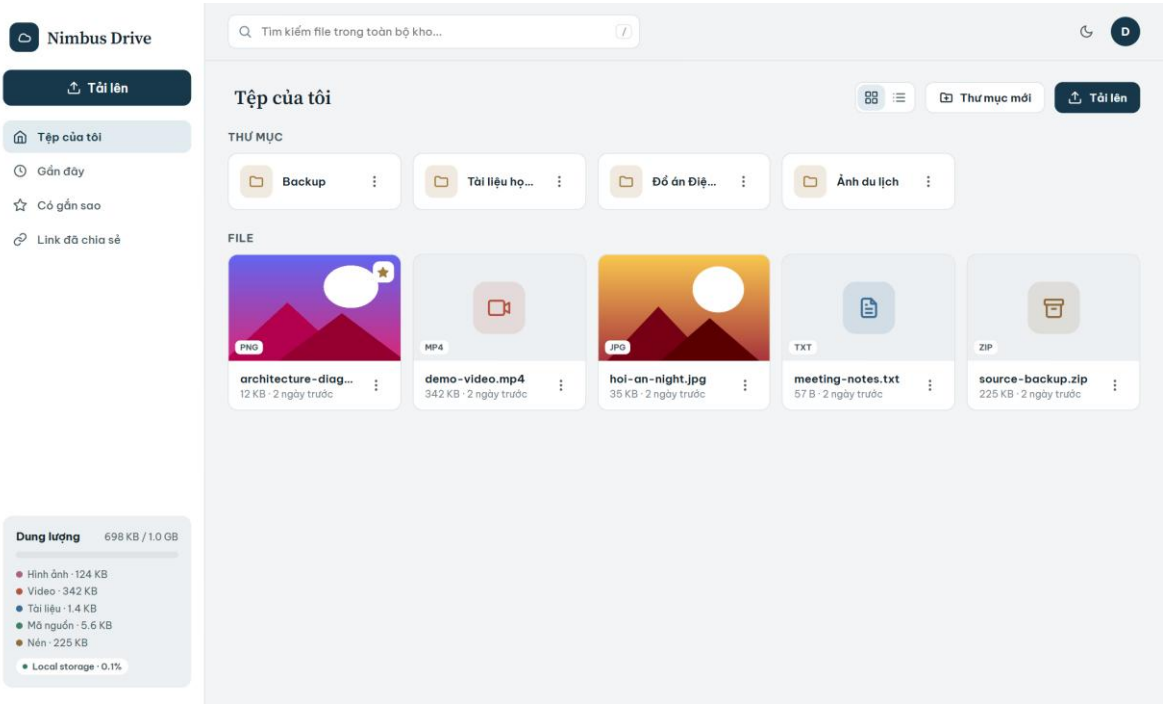
- **Nâng cấp:** mỗi lần push lên main, job deploy kéo image mới theo tag sha-<commit> và chạy lại docker compose up -d; Flyway tự áp dụng migration mới.
- **Rollback:** chạy lại workflow ở commit cũ (hoặc đặt APP_IMAGE về tag sha cũ trên server rồi docker compose up -d); dữ liệu không bị ảnh hưởng vì nằm ở volume PostgreSQL và bucket S3.
- **Tạm dừng để tiết kiệm chi phí:** đặt DEPLOY_ENABLED=false để dừng tự động deploy; có thể dừng (stop) EC2 hoặc terraform destroy khi không dùng. Lưu ý Elastic IP vẫn bị tính phí theo giờ dù máy đã dừng.
- **Dọn dẹp:** terraform destroy xóa toàn bộ tài nguyên; vì bucket bật versioning nên cần force_destroy_bucket = true nếu muốn xóa cả dữ liệu.

CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ

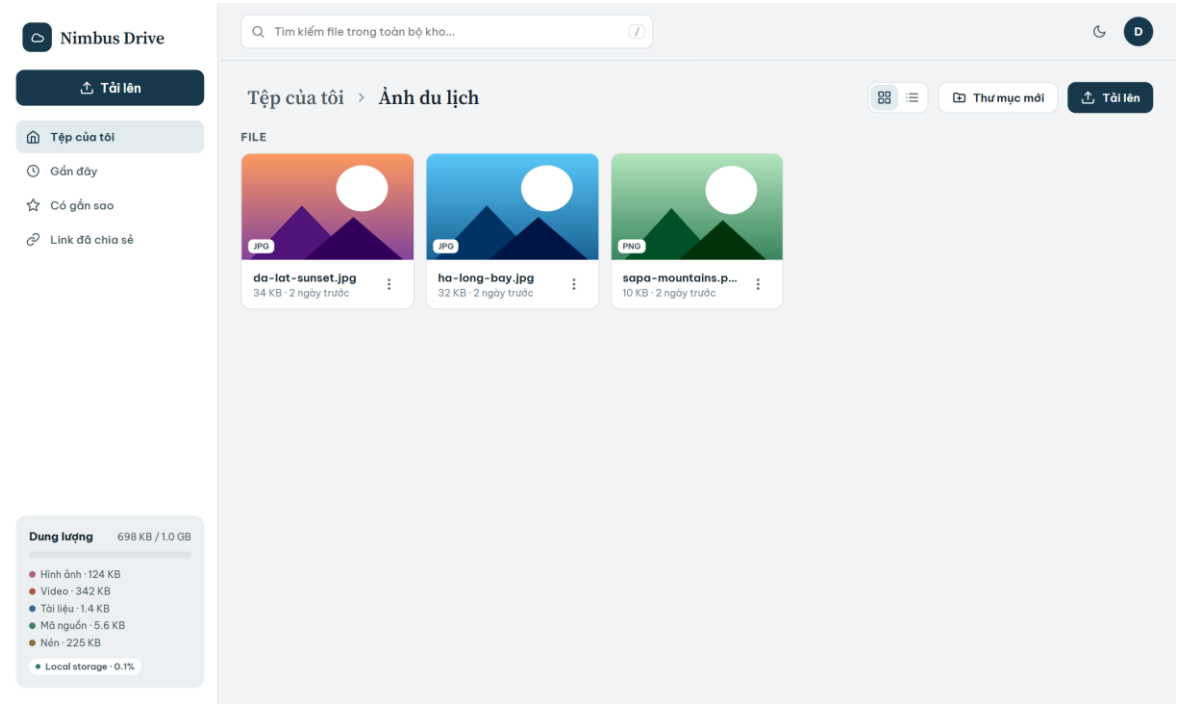
5.1. Giao diện ứng dụng



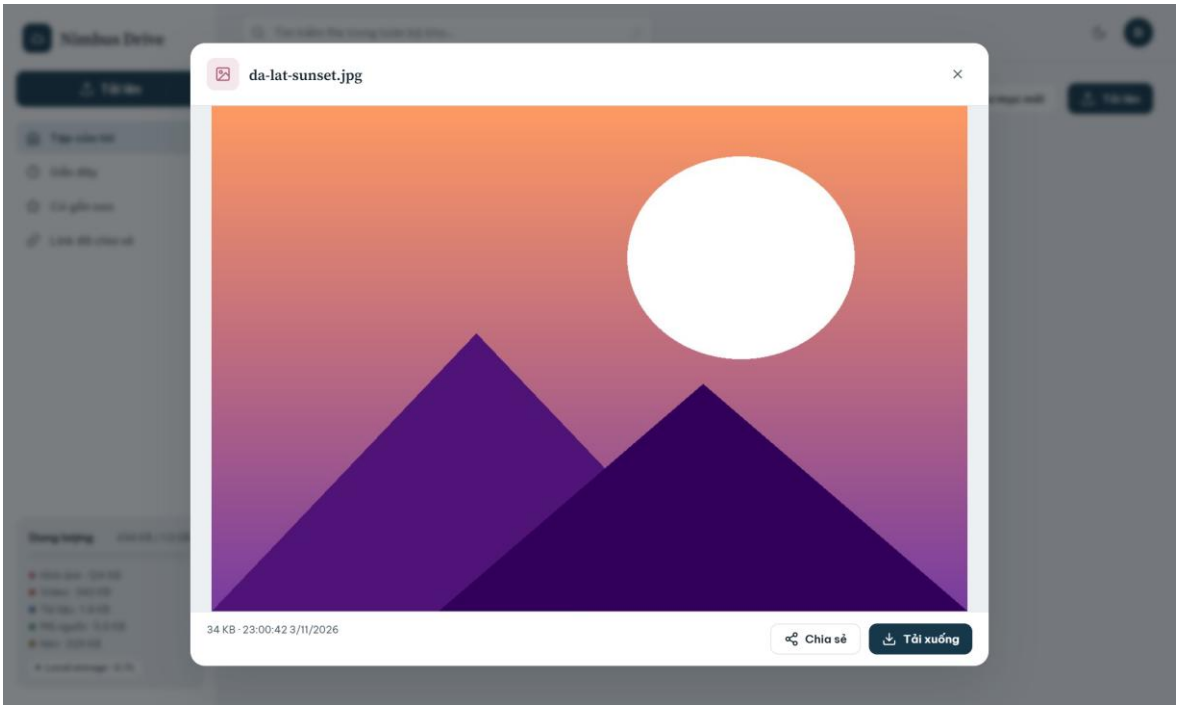
Hình 5.1. Trang đăng nhập / đăng ký



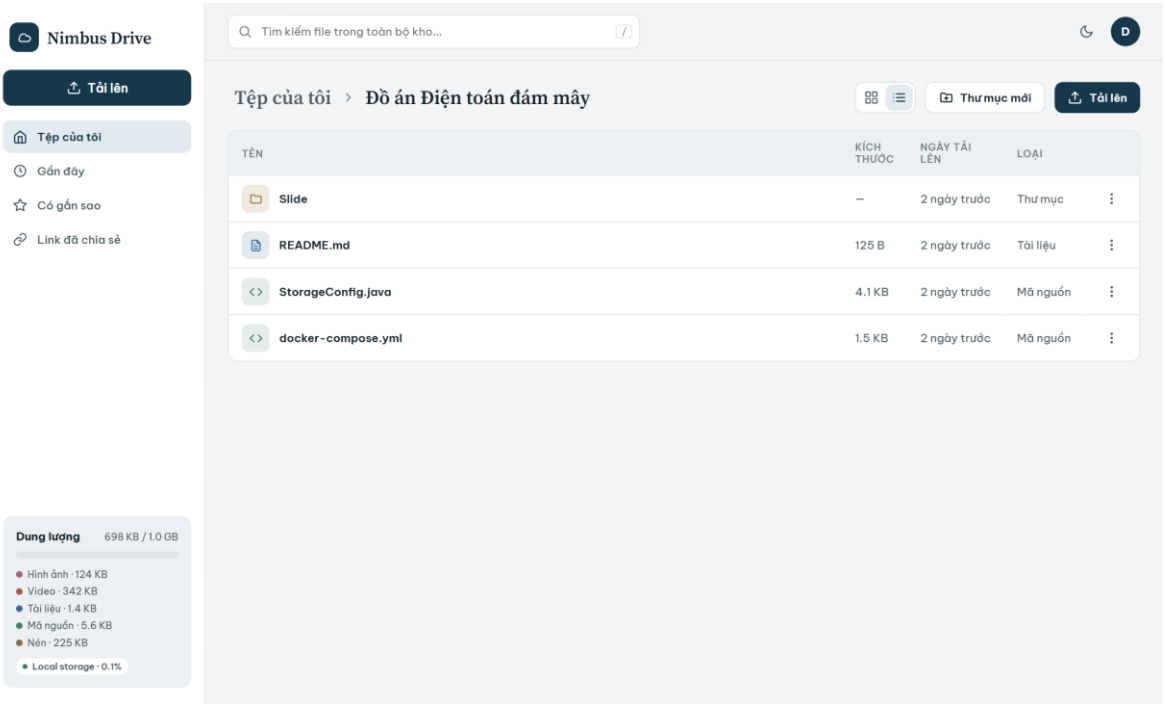
Hình 5.2. Trang quản lý file – chế độ lưới



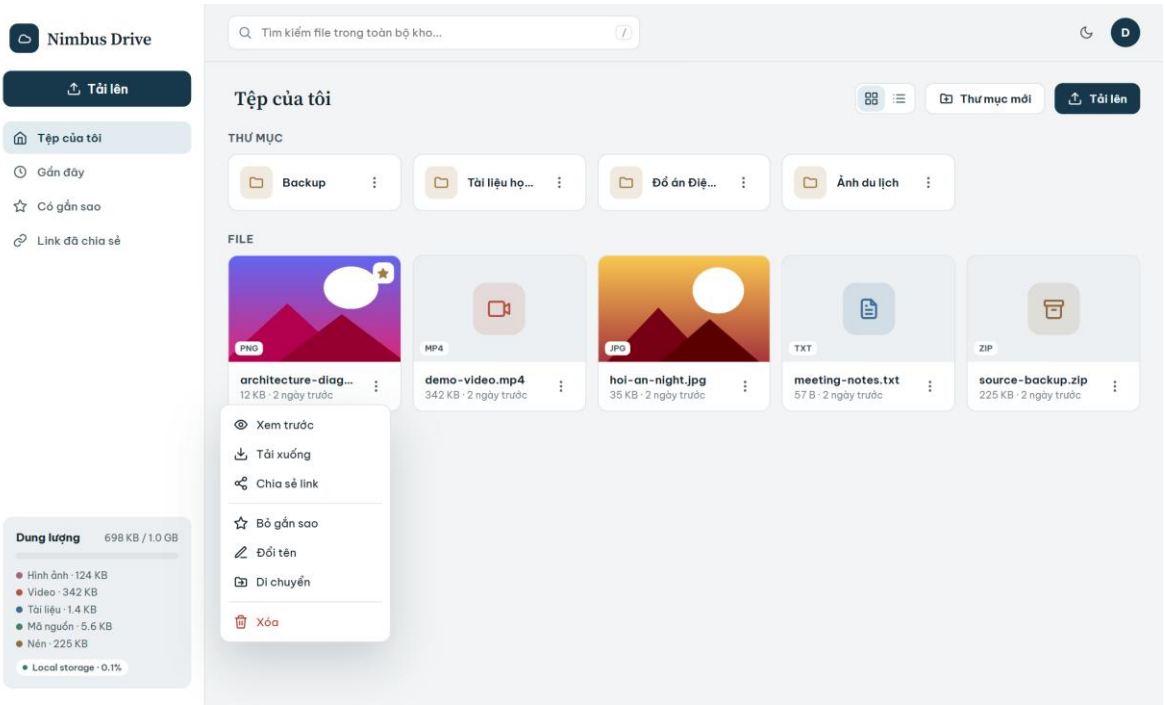
Hình 5.3. Thư mục ảnh với ảnh thu nhỏ



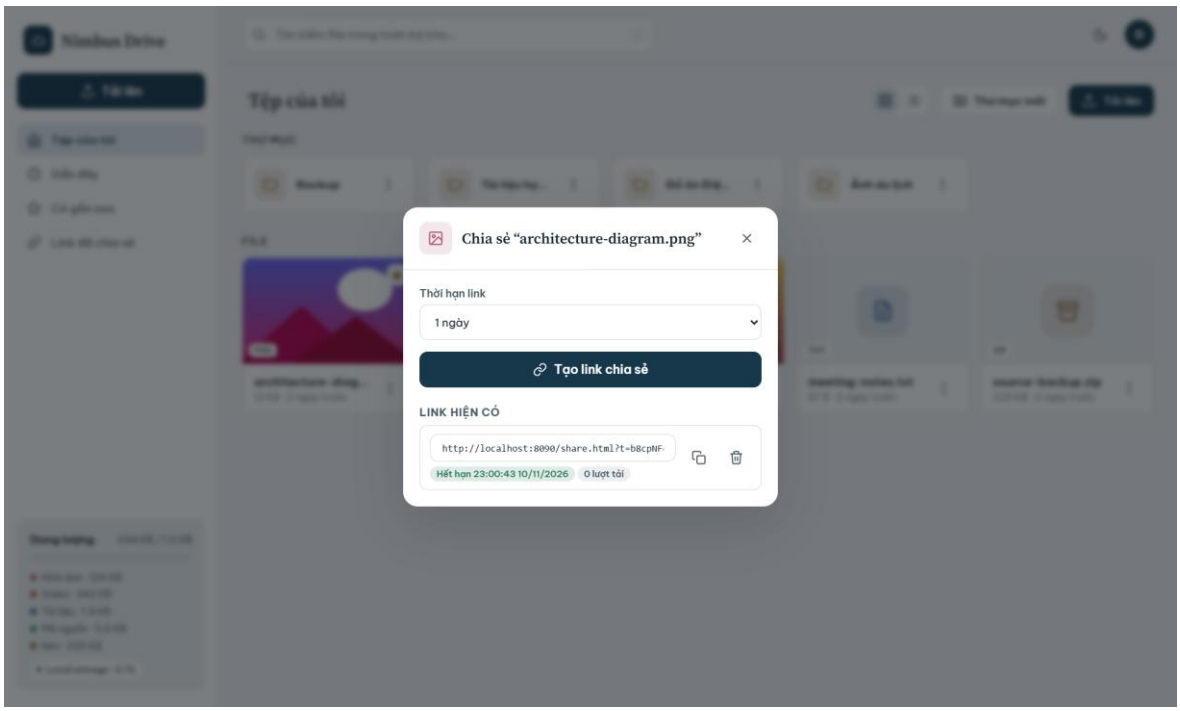
Hình 5.4. Xem trước ảnh trực tiếp



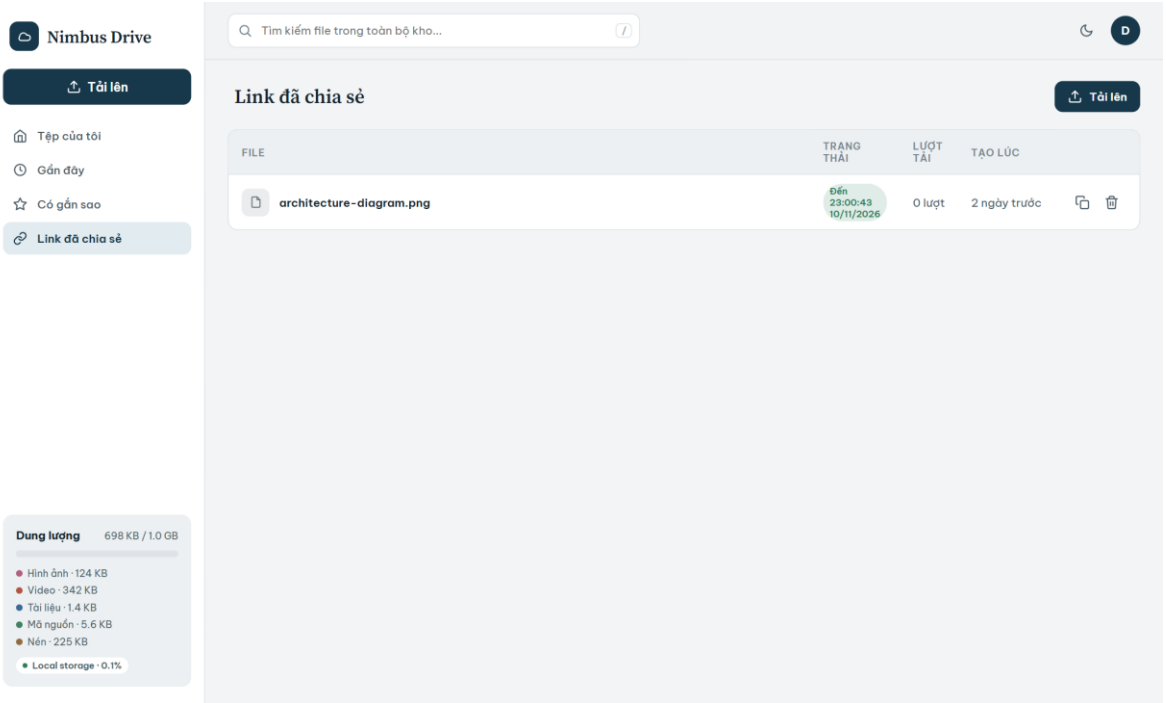
Hình 5.5. Chế độ danh sách và breadcrumb thư mục lồng nhau



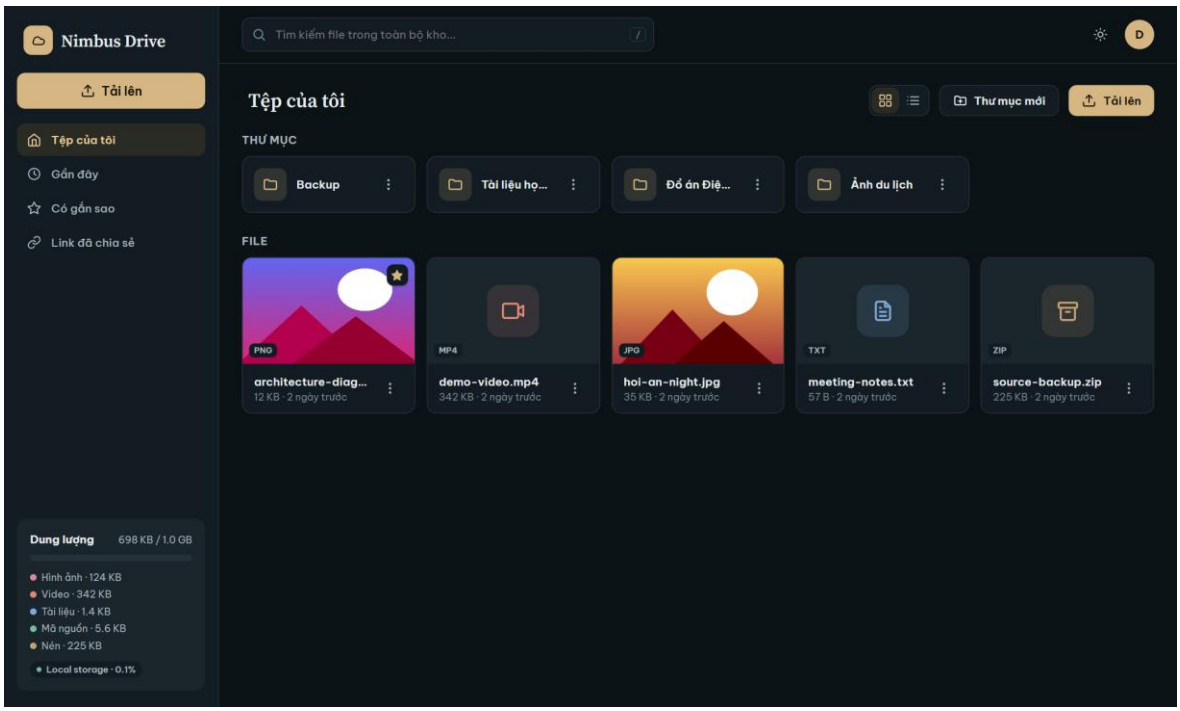
Hình 5.6. Menu thao tác trên file



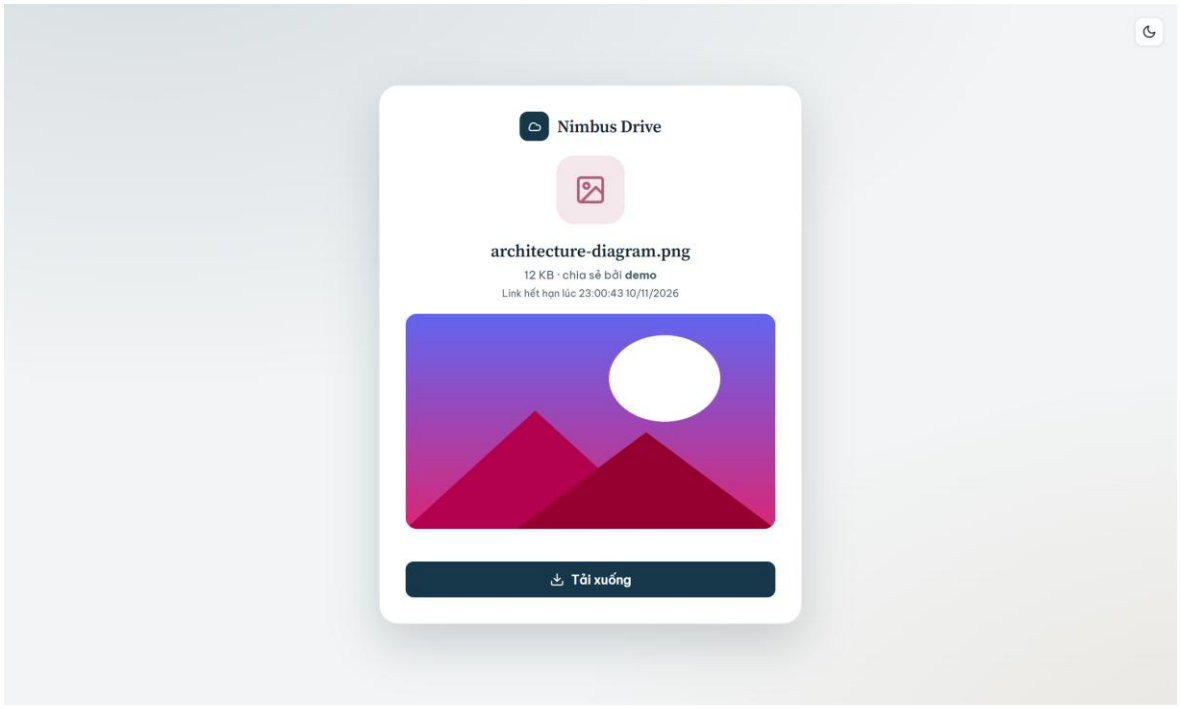
Hình 5.7. Hộp thoại tạo link chia sẻ có thời hạn



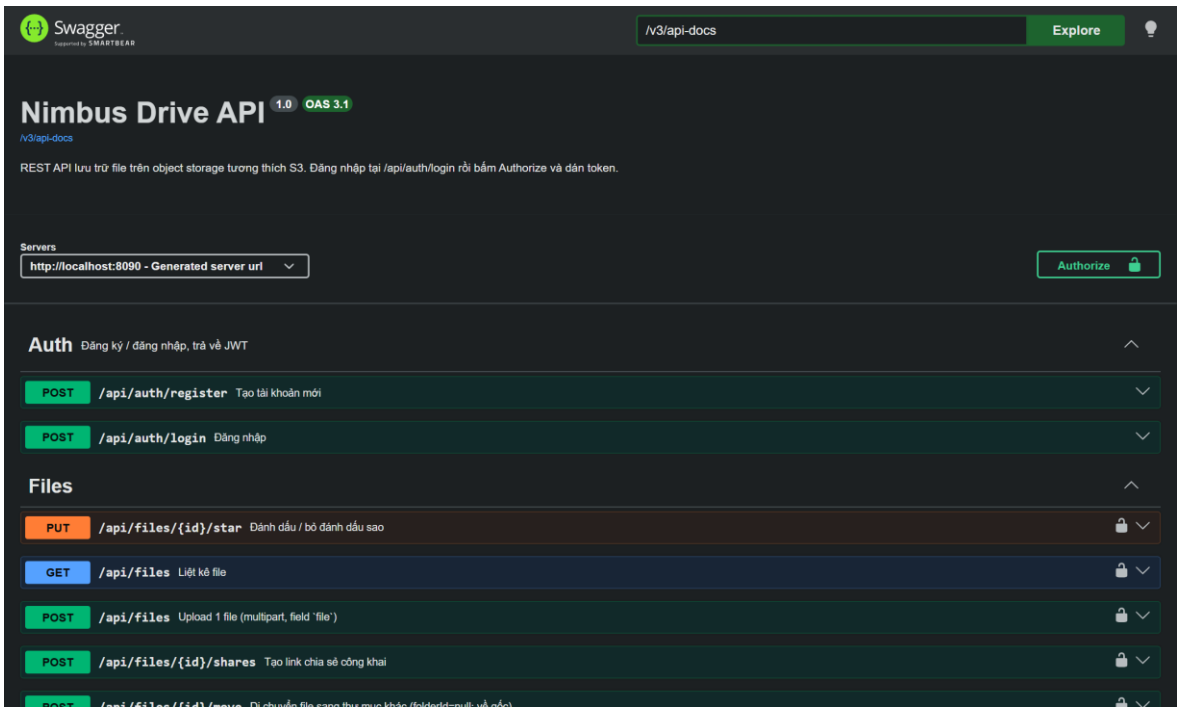
Hình 5.8. Quản lý các link đã chia sẻ



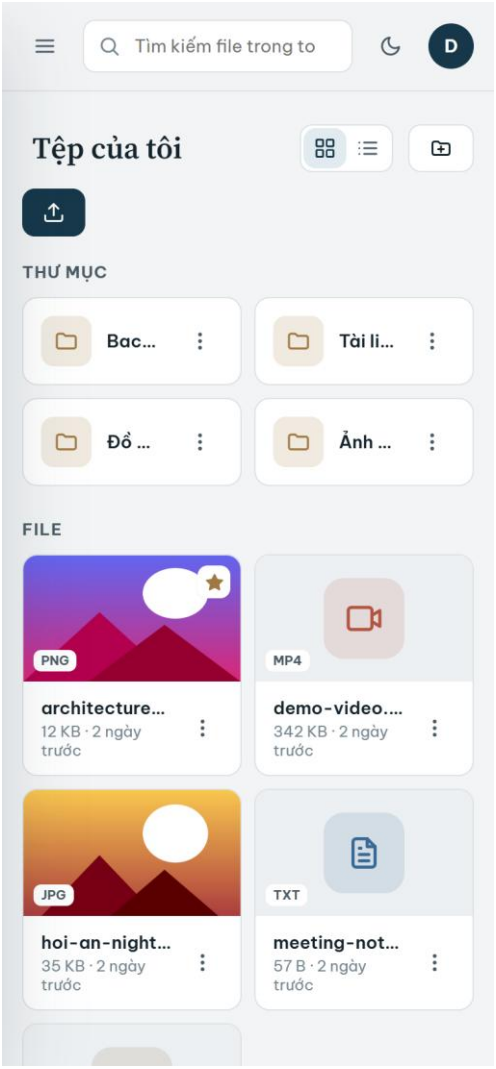
Hình 5.9. Giao diện tối (dark mode)



Hình 5.10. Trang tải file công khai cho khách



Hình 5.11. Tài liệu API Swagger UI



Hình 5.12. Giao diện trên điện thoại

5.2. Kiểm thử

Bảng 5.1. Kết quả kiểm thử tự động (mvn verify)

Bộ test	Số ca	Nội dung	Kết quả
JwtServiceTest	3	Sinh/giải mã token, sai khóa, hết hạn	Đạt
UnitTests	4	Làm sạch tên file, phân loại file, token chia sẻ, khóa đăng nhập	Đạt
ApiFlowTest (MockMvc + H2)	11	Bảo vệ endpoint, validate, trùng tài khoản, khóa đăng nhập, upload/tải/đổi tên/gắn sao/xóa, cô lập người dùng, quota, thư mục đệ quy, chia sẻ, chống XSS, presign, health	Đạt
PostgresMigrationIT	1	Flyway + JPA trên PostgreSQL 16 thật (Testcontainers)	Đạt (CI)
S3ObjectStorageIT	3	Put/Get/Delete, presigned URL, tạo bucket idempotent trên SeaweedFS thật	Đạt (CI)

22/22 ca kiểm thử đạt trên GitHub Actions: 18 ca unit/API và 4 ca integration với PostgreSQL 16 và SeaweedFS thật; độ phủ dòng lệnh của 18 ca chạy cục bộ đạt khoảng 80% (JaCoCo). Lần chạy CI đầu tiên cho thấy Testcontainers 1.19 không kết nối được Docker của runner nên 4 ca integration bị bỏ qua mà pipeline vẫn báo thành công. Nhóm đã nâng lên 1.21.4 và thêm bước làm pipeline thất bại nếu integration test bị bỏ qua.

Bảng 5.2. Danh sách 22 ca kiểm thử

Lớp test	Ca kiểm thử	Điều được kiểm chứng
JwtServiceTest	tokenRoundTripReturnsUsername	Token sinh ra giải mã lại đúng tên người dùng
	tokenSignedWithOtherSecretIsRejected	Token ký bằng khóa khác bị từ chối
	expiredTokenIsRejected	Token hết hạn bị từ chối
UnitTests	fileNamesStripPathsAndControlChars	Bỏ đường dẫn, ký tự điều khiển; từ chối tên rỗng, “.”, “..”
	categoriesFromTypeOrExtension	Phân loại hình ảnh/video/tài liệu/mã nguồn/nén theo MIME hoặc đuôi file
	shareTokensAreRandomAndUrlSafe	Token chia sẻ khác nhau mỗi lần và chỉ chứa ký tự an toàn cho URL
	loginLockResetsOnSuccess	Bộ đếm đăng nhập sai được xóa sau khi đăng nhập đúng
ApiFlowTest	protectedEndpointsRequireToken	Endpoint nghiệp vụ trả 401 khi không có token
	registerValidatesInputAndRejectsDuplicates	Validate username/mật khẩu; trùng tài khoản trả 409

	loginIsLockedAfterRepeatedFailures	Sai 5 lần → 429, đúng mật khẩu vẫn bị khóa trong thời gian khóa
	uploadListDownloadRenameDelete	Vòng đời đầy đủ của một file và nội dung tải về khớp bản gốc
	otherUsersCannotSeeMyFiles	Người dùng khác nhận 404 khi truy cập file của tôi
	quotaIsEnforced	Upload vượt quota trả 413
	foldersNestMoveAndDeleteRecursively	Thư mục lồng nhau, di chuyển file, xóa đệ quy cả file bên trong
	publicShareLinkWorksUntilRevoked	Khách tải được qua link, đếm lượt, sau khi thu hồi trả 404
	htmlIsNeverServedInline	File HTML bị ép tải về dù yêu cầu inline (chống XSS)
	presignIsNotAvailableOnLocalBackend	Backend local trả 501 khi xin presigned URL
	healthAndApiDocsArePublic	/actuator/health và Swagger truy cập được không cần đăng nhập
PostgresMigrationIT	schemaMatchesEntities	Flyway + Hibernate validate trên PostgreSQL 16 thật
S3ObjectStorageIT	putGetDeleteRoundTrip	Ghi/đọc/xóa object trên SeaweedFS thật
	presignedUrlDownloadsWithoutCredentials	Presigned URL tải được bằng HTTP thuần, không cần credential
	ensureBucketIsIdempotent	Tạo bucket nhiều lần không lỗi

5.3. Triển khai thực tế trên AWS

Hệ thống đã được triển khai thật bằng terraform apply tại vùng ap-southeast-2 (Sydney), tạo 17 tài nguyên; image do pipeline build được job Deploy đưa lên EC2 tự động. Kiểm thử đầu cuối qua Internet cho kết quả sau:

Bảng 5.3. Kết quả kiểm thử trên môi trường AWS

Kiểm tra	**Kết quả**
Đăng ký, upload, tải file qua HTTP công khai	Đạt, nội dung tải về khớp file gốc
Object xuất hiện trong bucket Amazon S3	Đạt, key dạng owner/uuid
App truy cập S3 bằng IAM role, không dùng access key	Đạt
Presigned URL tải trực tiếp từ S3	Đạt
Link chia sẻ công khai không cần đăng nhập	Đạt

Xóa file thì object biến mất khỏi danh sách S3	Đạt
Gọi API không có token / truy cập /actuator/prometheus từ ngoài	401 / 404 đúng thiết kế

Lần triển khai này cũng phát hiện một lỗi cấu hình: với IMDSv2, giới hạn hop mặc định bằng 1 khiến container không lấy được credential của IAM role; đã sửa bằng http_put_response_hop_limit = 2.

5.4. Đánh giá

Bảng 5.4. Đối chiếu mục tiêu

Mục tiêu	Kết quả
Ứng dụng lưu trữ file đầy đủ chức năng	Hoàn thành F01–F08, giao diện responsive, sáng/tối
Kiến trúc tương thích S3	Chạy với SeaweedFS (local) và Amazon S3 (AWS) không đổi mã; hỗ trợ presigned URL
Container hóa	Dockerfile multi-stage, layered, non-root; Compose 5 service với healthcheck và profile
CI/CD tự động	Pipeline test → image → scan → push → deploy → smoke test
IaC trên AWS	Terraform cho S3, IAM, EC2, SG, EIP theo nguyên tắc quyền tối thiểu
Bảo mật, giám sát	JWT, BCrypt, rate limit, CSP, chống IDOR/XSS; Actuator, Prometheus, Grafana

Ưu điểm

- Tách biệt rõ metadata và nội dung, thư mục ảo giúp thao tác tổ chức file không tốn chi phí trên storage.
- Không phụ thuộc nhà cung cấp (vendor-neutral) nhờ port/adapter và chuẩn S3 – đã chứng minh khi thay MinIO bằng SeaweedFS.
- Có thể chạy ở ba chế độ: dev không Docker (H2 + ổ đĩa), Docker Compose self-host, AWS.
- Quy trình phát hành tự động, có kiểm thử, quét bảo mật và khả năng rollback theo commit.

Hạn chế

- Upload/tải trong giao diện đi qua máy chủ ứng dụng (giới hạn 100 MB/file); chưa dùng multipart upload trực tiếp lên S3.
- Bộ đếm chống dò mật khẩu lưu trong RAM nên chưa dùng chung khi chạy nhiều instance.

- PostgreSQL chạy trong container trên cùng EC2 – một điểm lỗi đơn; chưa có backup tự động.
- Quét Trivy ban đầu báo 65 lỗ hổng HIGH/CRITICAL; sau khi nâng Spring Boot 3.3 lên 3.5 và các thư viện liên quan còn 2 lỗi CRITICAL ở spring-webmvc 6.2, chỉ có bản vá trên Spring 7 (Spring Boot 4). Pipeline hiện chỉ cảnh báo, chưa chặn.
- Bản triển khai trên AWS chỉ dùng HTTP qua địa chỉ IP vì chưa gắn tên miền, nên chưa có HTTPS.

5.5. Các sự cố gặp phải và cách xử lý

Quá trình thực hiện phát sinh nhiều sự cố thật; phần lớn chỉ lộ ra khi chạy trên môi trường thật hoặc trên CI. Bảng dưới đây ghi lại nguyên nhân gốc và cách khắc phục, là phần giá trị nhất để rút kinh nghiệm cho các dự án cloud sau này.

Bảng 5.5. Nhật ký sự cố và bài học

Sự cố	Nguyên nhân gốc	Cách khắc phục	Bài học
Không tải được MinIO	MinIO community ngừng phát hành bản dựng (dl.min.io trả 410 Gone)	Chuyển sang SeaweedFS 4.52, chỉ đổi cấu hình, không sửa mã nghiệp vụ	Trừu tượng hóa nhà cung cấp bằng chuẩn mở có giá trị thực tế
CI xanh nhưng integration test không chạy	Testcontainers 1.19 không kết nối được Docker của runner nên tự bỏ qua 4 ca	Nâng lên 1.21.4; thêm bước kiểm tra skipped = 0 trong failsafe-summary.xml	Kiểm thử bị bỏ qua phải làm pipeline đỏ, không được âm thầm xanh
App trong container không lấy được credential IAM	IMDSv2 mặc định hop limit = 1, container cần thêm 1 hop	http_put_response_hop_limit = 2 trong Terraform	Thử trên môi trường thật sớm; mặc định bảo mật có thể phá giả định
Server không kéo được image từ GHCR	Job deploy thiếu quyền packages: read; image mặc định private	Thêm permissions packages: read và docker login bằng GITHUB_TOKEN trong script deploy	Ghi rõ quyền tối thiểu cho từng job
Trivy báo 65 lỗ hổng HIGH/CRITICAL	Spring Boot 3.3 và các thư viện cũ (Tomcat, Netty, Jackson, PostgreSQL JDBC, httpcore5)	Nâng Spring Boot 3.5.16, ghi đè phiên bản các thư viện bị ảnh hưởng, chạy apt upgrade ở image chạy	Còn 2 lỗi CRITICAL của spring-webmvc 6.2 chỉ vá ở Spring 7 → ghi nhận như rủi ro còn lại
Giao diện chưa nhất quán	Giao diện ban đầu dùng bộ màu indigo – hồng, thiếu phân cấp thị giác	Thiết kế lại bằng biến CSS: bảng màu mực – sâm panh, phông có chân cho tiêu đề, chuyển sáng/tối	Dùng design token ngay từ đầu để đổi giao diện không đụng logic

5.6. Chi phí vận hành

Bảng 5.6. Các thành phần chi phí khi chạy trên AWS

Thành phần	Cách tính	Cách tối ưu
EC2 t3.small	Theo giờ chạy (on-demand)	Dừng máy khi không demo; dùng Savings Plan nếu chạy lâu dài
EBS gp3 20 GB	Theo GB-tháng, tính cả khi máy dừng	Giảm dung lượng ổ nếu không cần; xóa khi terraform destroy
Elastic IP / IPv4 công khai	Từ 02/2024 AWS tính phí theo giờ cho mọi địa chỉ IPv4 công khai, kể cả đang gắn máy	Giải phóng cùng terraform destroy khi không dùng
Amazon S3	Theo GB lưu trữ, số request và dữ liệu ra Internet	Lifecycle xóa phiên bản cũ; presigned URL giảm lưu lượng qua EC2
GitHub Actions, GHCR	Miễn phí trong hạn mức repo công khai	Bật cache Buildx (type=gha) để rút ngắn thời gian build

Đơn giá cụ thể thay đổi theo vùng và thời điểm nên đồ án không nêu con số; có thể tính bằng AWS Pricing Calculator với cấu hình trên. Với một máy t3.small và vài GB dữ liệu, chi phí chủ yếu đến từ EC2 và EBS, nên việc dừng hoặc hủy hạ tầng khi không sử dụng là biện pháp tiết kiệm hiệu quả nhất.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đồ án đã thiết kế và triển khai thành công **Nimbus Drive** – nền tảng lưu trữ file đám mây đóng gói container, xây dựng trên Spring Boot và kiến trúc tương thích Amazon S3, kèm pipeline CI/CD tự động hoàn chỉnh và hạ tầng AWS dưới dạng mã. Qua đó, em đã vận dụng các kiến thức cốt lõi của điện toán đám mây: mô hình IaaS/PaaS/SaaS, lưu trữ đối tượng, container hóa, quản lý định danh và truy cập (IAM), tự động hóa triển khai và giám sát hệ thống.

6.2. Hướng phát triển

Bảng 6.1. Đề xuất phát triển tiếp

Hướng	Mô tả	Ưu tiên
Upload trực tiếp lên S3	Presigned PUT + S3 multipart upload: file nhiều GB, giảm tải server, resume khi mất mạng	Cao
Thùng rác & phiên bản	Xóa mềm, khôi phục trong 30 ngày; tận dụng S3 versioning để xem/khôi phục phiên bản cũ	Cao
Hạ tầng managed	Amazon RDS (Multi-AZ, backup tự động), ECS Fargate/EKS + ALB, CloudFront CDN	Trung bình
Chia sẻ nội bộ	Chia sẻ cho user khác với quyền xem/sửa, link có mật khẩu, giới hạn lượt tải	Trung bình
Bảo mật nâng cao	Refresh token, đăng nhập Google (OAuth2/OIDC), 2FA, quét virus ClamAV, rate limit bằng Redis	Trung bình
Trải nghiệm	Upload cả thư mục, thumbnail sinh phía server (S3 event + Lambda), tìm kiếm toàn văn	Thấp
Vận hành	Cảnh báo Alertmanager, log tập trung (Loki/CloudWatch), staging environment, blue-green deploy	Thấp

6.3. Bài học kinh nghiệm

- **Chuẩn mở là bảo hiểm:** việc bám chuẩn S3 và tách port/adapter giúp đổi MinIO sang SeaweedFS và chuyển giữa local/AWS mà không sửa nghiệp vụ.
- **Thứ tự thao tác quan trọng hơn giao dịch phân tán:** với hai kho dữ liệu, chọn đúng thứ tự ghi/xóa đơn giản và hiệu quả hơn so với cố gắng làm giao dịch hai pha.
- **Kiểm thử phải chạy được trên môi trường giống thật:** lỗi hop limit của IMDSv2 hay lỗi Testcontainers bị bỏ qua chỉ lộ ra khi chạy trên AWS hoặc CI.
- **Bảo mật là quá trình liên tục:** quét Trivy cho thấy rủi ro thay đổi theo từng tuần; cần quy trình nâng cấp phụ thuộc định kỳ thay vì sửa một lần.
- **Hạ tầng dưới dạng mã giúp tái tạo và dọn dẹp:** toàn bộ môi trường AWS dựng lên và hủy đi bằng một lệnh, rất phù hợp với bài toán học tập có ngân sách hạn chế.

TÀI LIỆU THAM KHẢO

[1] P. Mell, T. Grance, "The NIST Definition of Cloud Computing," NIST Special Publication 800-145, 2011.

[2] Amazon Web Services, "Amazon S3 User Guide," docs.aws.amazon.com/AmazonS3.

[3] Amazon Web Services, "Authenticating Requests (AWS Signature Version 4)" và "Sharing objects with presigned URLs".

[4] Amazon Web Services, "AWS SDK for Java 2.x Developer Guide".

[5] VMware Tanzu, "Spring Boot Reference Documentation," phiên bản 3.3.

[6] Spring, "Spring Security Reference" – Authentication, Headers (Content Security Policy).

[7] Docker Inc., "Dockerfile reference" và "Docker Compose specification".

[8] GitHub, "GitHub Actions documentation" – Workflow syntax, Publishing Docker images.

[9] HashiCorp, "Terraform AWS Provider documentation".

[10] SeaweedFS, "Amazon S3 API" – github.com/seaweedfs/seaweedfs/wiki.

[11] M. Nottingham, E. Wilde, "RFC 7807 – Problem Details for HTTP APIs," IETF, 2016.

[12] OWASP Foundation, "OWASP Top 10" và "File Upload Cheat Sheet".

[13] A. Wiggins, "The Twelve-Factor App," 12factor.net.

PHỤ LỤC: HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY

A. Chạy bằng Docker (đầy đủ):

```
cp .env.example .env          # đổi mật khẩu, JWT_SECRET
docker compose up -d --build
# Web: http://localhost:8080   Swagger: /swagger-ui.html   S3: http://localhost:8333
docker compose --profile monitoring up -d   # Grafana http://localhost:3000
```

B. Chạy chế độ dev không cần Docker (H2 + lưu file trên ổ đĩa):

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev     # Windows: mvnw.cmd
```

C. Kiểm thử:

```
./mvnw verify      # báo cáo độ phủ: target/site/jacoco/index.html
```

D. Bật triển khai tự động: tạo repo variable DEPLOY_ENABLED=true, APP_URL; secrets DEPLOY_HOST, DEPLOY_USER, DEPLOY_SSH_KEY; sau đó mỗi lần push lên nhánh main, pipeline tự build, kiểm thử và triển khai.

E. Xử lý sự cố thường gặp:

Bảng 6.2. Bảng xử lý sự cố

Hiện tượng	Nguyên nhân thường gặp	Cách kiểm tra / khắc phục
App không khởi động, log "Connection refused" tới storage	SeaweedFS/S3 chưa sẵn sàng hoặc sai S3_ENDPOINT	docker compose logs seaweedfs; ứng dụng tự thử lại 10 lần × 3 giây
Upload trả 502	Không kết nối được S3 hoặc sai credential/bucket	Kiểm tra S3_BUCKET, S3_REGION, IAM role (trên AWS), docker compose logs app
Tải bằng presigned URL bị lỗi trong trình duyệt	S3_PUBLIC_ENDPOINT trỏ tới địa chỉ trình duyệt không truy cập được; thiếu CORS	Đặt S3_PUBLIC_ENDPOINT đúng địa chỉ công khai; kiểm tra cấu hình CORS của bucket
Trên EC2 app không lấy được credential	IMDSv2 hop limit = 1	Đặt http_put_response_hop_limit = 2 rồi terraform apply
Deploy thất bại ở bước pull image	Image private, server chưa đăng nhập GHCR	Kiểm tra quyền packages: read; docker login ghcr.io
Web không truy cập được từ Internet	Instance dừng, Security Group, hoặc DEPLOY_ENABLED chưa bật	aws ec2 describe-instances; kiểm tra SG mở 80/443; curl /actuator/health
Đăng nhập báo 429	Sai mật khẩu 5 lần liên tiếp	Chờ 5 phút hoặc khởi động lại ứng dụng (bộ đếm lưu trong RAM)

F. Các cổng và địa chỉ mặc định khi chạy Docker Compose:

Bảng 6.3. Cổng dịch vụ

Dịch vụ	Địa chỉ	Ghi chú
Ứng dụng web / API	http://localhost:8080	Trên AWS chỉ lắng nghe 127.0.0.1, truy cập qua Caddy
Swagger UI	http://localhost:8080/swagger-ui.html	Tài liệu OpenAPI tại /v3/api-docs
S3 (SeaweedFS)	http://localhost:8333	Cần cho presigned URL từ trình duyệt
Prometheus	http://localhost:9090	Profile monitoring
Grafana	http://localhost:3000	Profile monitoring; đổi mật khẩu quản trị mặc định
Caddy	80 / 443	Profile proxy; đặt DOMAIN để có HTTPS